Universidad Católica San Pablo

FACULTAD DE INGENIERÍA Y COMPUTACIÓN

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

Escuela Profesional de Ciencia de la Computación

Arquitectura basada en representación intermedia para la generación confiable de consultas SQL/PGQ desde lenguaje natural

Tesis

Presentada por el bachiller

Guido Luis Tapia Oré

Para Optar por el Título profesional de:

Licenciado en Ciencia de la Computación

Asesor: Dra. Regina Paola Ticona Herrera

** Arequipa, Julio 2026 **

A mi familia por su incondicional apoyo en este proceso, a los profesores que fueron una guía en este proceso de investigación, a mis compañeros que ayudaron a no rendirme en este proceso.

Abreviaturas

LLM Large Language Model

PGQ Property Graph Queries

RBAC Role-Based Access Control

RDBMS Relational Database Management System

RI Representación Intermedia

SQL Structured Query Language

Agradecimientos

Aquí deberás colocar a quien y porque agradeces. Ejemplo:

En primer lugar deseo agradecer a Dios por haberme guiado a lo largo de estos cinco años de estudio.

Agradezco a mis padres por el apoyo brindado para forjarme como un profesional.

Agradezco a la universidad, mi *alma matter*, por haberme cobijado y brindado la formación que ahora me permitirá ayudar a construir una mejor sociedad.

Agradezco de forma muy especial a mi asesor Prof. Dr./Mag. nombre 1 por haberme guiado en esta tesis. ...

Deseo agradecer de forma especial a mis profesores: nombre 1, nombre 2, nombre 3 porque fueron ejemplos que deseo seguir en mi vida profesional.

Deseo agradecer al personal administrativo de la universidad: nombre 1, nombre 2, nombre 3. Muchas gracias por la atención brindada y porque siempre estuvieron dispuestas a ayudarnos.

Resumen

Aquí deberás colocar hasta 300 palabras como máximo, describiendo el problema que intentas resolver, la justificación, aportes o soluciones que planteas, qué tanto los haz alcanzado en calidad de resultados obtenidos.

Abstract

Here you should enter up to 300 words maximum, describing the problem you are trying to solve, the justification, contributions or solutions you are proposing, how much you have achieved them in terms of obtained outcomes.

Índice general

1. Introducción **2**

1.1. Motivación y Contexto 2

1.2. Planteamiento del Problema 4

1.3. Objetivos 5

1.3.1. Objetivos Específicos 5

1.4. Organización de la tesis 5

1.5. Cronograma 6

2. Fundamentos de Generación Text-to-Query y Consultas Híbridas **8**

2.1. Modelos de lenguaje y generación estructurada 8

2.1.1. Arquitectura transformer y decodificación autorregresiva 9

2.1.2. Prompting y aprendizaje en contexto 10

2.2. Modelos de datos y lenguajes de consulta 10

2.2.1. Modelo relacional y álgebra relacional 10

2.2.2. SQL como lenguaje declarativo 11

2.2.3. Grafos de propiedad y pattern matching 11

2.2.4. El estándar SQL/PGQ como integración 12

2.3. Semantic parsing y generación *text-to-query* 12

2.3.1. *Semantic parsing*: definición y marco general 12

2.3.2. *Text-to-SQL* como caso particular y extensión a *text-to-query* 12

2.3.3. *Schema linking* y *grounding* al esquema 13

2.4. Representaciones intermedias 13
2.4.1. Noción formal de representación intermedia 13
2.4.2. Propiedades deseables 14
2.4.3. Separación entre generación estadística y compilación determinista . 14
2.5. Gramáticas formales y decodificación restringida 14
2.5.1. Gramáticas formales y jerarquía de Chomsky 14
2.5.2. Decodificación restringida como acoplamiento generación–parser . . 15
2.5.3. Sistemas de tipos y reglas locales de validación 15
2.5.4. Alcance y límites formales 16
2.6. Verificación de consultas generadas 16
2.6.1. Verificación estática 16
2.6.2. Verificación dinámica y oráculos parciales 16
2.6.3. Testing metamórfico 17
2.6.4. Bucles de refinamiento 17

3. Arquitecturas y Enfoques Actuales en Text-to-Query 18
3.1. Marco taxonómico de análisis 18
3.2. Paradigma de Aprendizaje 21
3.2.1. Aprendizaje con parámetros congelados 21
3.2.2. Aprendizaje con parámetros ajustados 22
3.2.3. Estrategias híbridas 23
3.2.4. Observaciones críticas 23
3.3. Arquitectura del Sistema 23
3.3.1. Arquitecturas monolíticas 24
3.3.2. Pipelines modulares 24
3.3.3. Arquitecturas multiagente 24
3.3.4. Arquitecturas agentic-search 25
3.3.5. Arquitecturas de generación con reordenamiento 25

3.3.6. Observaciones críticas 25

3.4. Estrategias de Representación 26

3.4.1. Codificación de la entrada 26

3.4.2. Representación intermedia 27

3.4.3. Estrategias de decodificación 28

3.4.4. Observaciones críticas 28

3.5. Mecanismos de Enriquecimiento Contextual 29

3.5.1. Schema linking 29

3.5.2. Recuperación de contenido de la base de datos 29

3.5.3. Conocimiento externo y de dominio 30

3.5.4. Auto-refinamiento 30

3.5.5. Observaciones críticas 30

3.6. Paradigma de Razonamiento 30

3.6.1. Razonamiento directo 31

3.6.2. Cadena de pensamiento 31

3.6.3. Razonamiento descompositivo 31

3.6.4. Razonamiento de múltiples caminos 31

3.6.5. Razonamiento de retroalimentación iterativa 32

3.6.6. Observaciones críticas 32

3.7. Propiedades Evaluativas Objetivo 32

3.7.1. Corrección 32

3.7.2. Eficiencia 33

3.7.3. Robustez 33

3.7.4. Adaptabilidad 33

3.7.5. Escalabilidad 33

3.7.6. Interpretabilidad 34

3.7.7. Observaciones críticas 34

3.8. Síntesis y Brecha de Investigación 34

4. Propuesta: Arquitectura basada en Representación Intermedia para Generación Confiable de Consultas SQL/PGQ 36

4.1. Requisitos y principios de diseño 37

4.2. Flujo metodológico general 38

4.3. Arquitectura y componentes 40

4.4. Implementación sobre la base experimental 40

4.5. Justificación de decisiones de diseño 40

4.6. Síntesis y transición 40

5. Experimentos 41

6. Resultados 42

7. Conclusiones y Trabajos Futuros 43

7.1. Problemas encontrados 43

7.2. Recomendaciones 43

7.3. Trabajos futuros 43

Bibliografía 49

Índice de tablas

1.1. Cronograma de actividades de la tesis. 6

Índice de figuras

4.1. Flujo metodológico general del *pipeline* propuesto. Las cinco etapas secuenciales procesan la pregunta en lenguaje natural y el esquema dual (relacional y grafo de propiedad) hasta producir una consulta Structured Query Language (SQL)/Property Graph Queries (PGQ) verificada. 39

Capítulo 1

Introducción

Este capítulo establece el contexto, la motivación y el problema de investigación que aborda la presente tesis, cuyo enfoque central radica en el diseño de una arquitectura de razonamiento estructural, basada en Representación Intermedia (RI), para mitigar las alucinaciones en la generación *text-to-query* de consultas empresariales complejas, específicamente orientada al estándar híbrido SQL/PGQ.

Esta investigación se sitúa en la intersección entre los Large Language Models (LLMs), los sistemas de gestión de bases de datos relacionales (Relational Database Management Systems (RDBMSs)) y los lenguajes de consulta híbridos, pues estos lenguajes combinan el paradigma relacional tradicional con las consultas sobre grafos de propiedades (*property graphs*), un modelo recientemente formalizado en estándares internacionales como ISO/IEC [ISO/IEC JTC 1/SC 32, 2023].

Como se evidenciará en el posterior desarrollo, el diseño propuesto representa una respuesta estructurada al creciente interés de la industria por desplegar interfaces en lenguaje natural altamente fiables en entornos empresariales a gran escala. Reducir las alucinaciones y mantener la consistencia lógica constituyen retos críticos actuales [Chi, 2025, Qu et al., 2024], los cuales demandan métodos avanzados que integren el razonamiento tabular y de grafos [Stoisser et al., 2025].

1.1. Motivación y Contexto

En el contexto actual de la adopción de LLMs en procesos empresariales y el creciente interés por integrarlos en la optimización de flujos de trabajo, se evidencia una clara tendencia hacia interfaces basadas en lenguaje natural [Adedeji and Mabere, 2023], en detrimento de las interfaces gráficas tradicionales. Este cambio es particularmente notable en organizaciones *data-driven*, cuya toma de decisiones depende de la capacidad de responder preguntas de negocio en ciclos cortos y con altas garantías de calidad [Pérez et al., 2025]. En este marco, las interfaces en lenguaje natural para la consulta de datos han resurgido con fuerza, habilitando experiencias de *self-service analytics*. Estas permiten que perfiles no técnicos formulen preguntas en texto libre y lenguaje cotidiano,

obteniendo resultados estructurados sin la necesidad de escribir manualmente consultas complejas [Quamar et al., 2022].

Sin embargo, el despliegue de estos modelos en entornos de producción demanda el cumplimiento de tres pilares críticos: (i) *integridad técnica*, que garantiza tanto la ejecutabilidad en el RDBMS como la fidelidad semántica de la consulta [Lei et al., 2025]; (ii) *seguridad y gobernanza*, mediante mecanismos de control de acceso basado en roles (Role-Based Access Control (RBAC)) y auditoría para prevenir la exfiltración de datos sensibles [Liu et al., 2025]; y (iii) *robustez y escalabilidad del esquema*, asegurando que el modelo mantenga su precisión frente a catálogos de datos masivos, dinámicos y con nomenclaturas complejas [Tang et al., 2026].

Asimismo, a pesar de los avances acelerados, los LLMs aún presentan fallas sistemáticas de *fidelidad y grounding* al generar artefactos formales. Específicamente en la tarea de *text-to-query*, estas fallas se manifiestan como referencias a tablas o columnas inexistentes, rutas de *join* inventadas, condiciones lógicas mal interpretadas o agregaciones incongruentes con el esquema. Si bien esta es una limitación general de los modelos de lenguaje, el riesgo se agrava severamente cuando el código se genera para un RDBMS: un error no solo afecta la exactitud analítica, sino que puede inducir sobrecostos computacionales, vulnerabilidades de seguridad e incluso modificar datos existentes. Investigaciones recientes demuestran que el alto desempeño en *benchmarks* clásicos decae drásticamente al enfrentarse a bases de datos corporativas reales, ruidosas y con flujos de trabajo de múltiples pasos [Li et al., 2023, Lei et al., 2025], lo cual refuerza la necesidad de transitar desde enfoques de *prompting* monolíticos y arquitecturas puramente agénticas hacia sistemas más robustos y orquestados [Fürst et al., 2024].

A su vez en paralelo, el ecosistema de bases de datos está evolucionando hacia modelos híbridos. En dominios como prevención de fraude, ciberseguridad, logística y *compliance*, el modelado de relaciones mediante grafos resulta más natural para expresar caminos y vecindarios complejos. No obstante, dado que las bases de datos relacionales siguen dominando el entorno corporativo por su estabilidad, la integración de *property graphs* se presenta como una evolución natural de la industria. Esto se valida con la reciente estandarización ISO/IEC 9075-16:2023 (SQL/PGQ), la cual formaliza la consulta de grafos de propiedades directamente desde SQL [ISO/IEC JTC 1/SC 32, 2023]. Si bien esto unifica ambos paradigmas, también incrementa significativamente la dificultad de la tarea *text-to-query*: el espacio de búsqueda se expande y la validación lógica se vuelve más rigurosa, multiplicando el riesgo de alucinaciones.

En la industria actual, las soluciones comerciales suelen apoyarse en estrategias heurísticas como inyectar el esquema en el *prompt* o aplicar ciclos de autocorrección mediante retroalimentación en texto libre. Estos métodos han demostrado ser frágiles ante la ambigüedad semántica del usuario y el *schema drift* [Ding et al., 2026]. Por su parte, la literatura reciente aborda este reto desde diversos frentes: decodificación restringida por gramática para garantizar validez sintáctica [Scholak and et al., 2022], arquitecturas modulares que dividen la carga de razonamiento [Pourreza and Rafiei, 2023], técnicas de alineación pre-generación para mitigar alucinaciones [Qu et al., 2024], métodos de evaluación sin *ground truth* como el *metamorphic testing* [Chi, 2025], y la unificación del razonamiento tabular y de grafos [Stoisser et al., 2025], entre otros.

Estos antecedentes convergen en una observación central: si bien existen técnicas parciales para mitigar alucinaciones en *text-to-SQL* y multiples propuestas dentro de esta área, aún existe un gran campo para investigar su implementación de forma sistémica y mejora, especialemente orientdo a la integración con el estándar híbrido SQL/PGQ. La ausencia de una arquitectura que coordine estas técnicas sobre un esquema dual (relacional y de grafo) deja sin resolver el problema de fidelidad en entornos empresariales donde coexisten ambos paradigmas. Es esta brecha la que constituye la principal motivación de la presente investigación.

1.2. Planteamiento del Problema

La generación de consultas a bases de datos a partir de lenguaje natural exige resolver un problema de traducción entre dos espacios formalmente distintos: el espacio continuo y ambiguo del texto libre y el espacio discreto de un lenguaje de consulta con semántica determinista, restringido por un esquema de base de datos específico. En entornos empresariales, donde las decisiones operativas dependen de la corrección de estas consultas, el problema adquiere relevancia práctica directa, dado que una consulta incorrecta no solo produce resultados erróneos, sino que puede traducirse en sobrecostos computacionales y comprometer la integridad de los datos.

En el contexto de la creciente integración de soluciones basadas en LLMs, estos modelos, al operar mediante predicción autorregresiva de tokens, carecen de mecanismos intrínsecos que vinculen la secuencia generada con las restricciones formales del esquema subyacente ni con la semántica específica del lenguaje de consulta objetivo. Esta desalineación entre la generación estadística y la corrección formal constituye una de las causas centrales de las denominadas *alucinaciones* en *text-to-query*: los modelos generan secuencias plausibles pero semánticamente inválidas respecto al esquema. En la práctica, esto se manifiesta como referencias a tablas o columnas inexistentes, uniones inválidas, condiciones lógicas inconsistentes o patrones de grafo mal formados.

El problema se agrava al considerar el estándar híbrido SQL/PGQ, donde la generación debe operar simultáneamente sobre dos estructuras formales superpuestas: el esquema relacional, el cual incluye tablas, columnas, restricciones, y el grafo de propiedad que representaría los nodos, aristas, propiedades clave-valor. En este contexto, los errores no solo afectan a cada estructura por separado, sino que surgen nuevas categorías de fallo derivadas de la incoherencia cruzada entre ambas. Esta dualidad contribuye a incrementar significativamente el espacio de decisiones del modelo y, con ello, la dificultad de garantizar la corrección de las consultas generadas.

Si bien la literatura reciente ha propuesto técnicas parciales para abordar distintas facetas de este problema, lo que se evidencia es un claro declive de rendimiento de las propuestas cuando son evaluadas con datasets de consultas híbridas SQL/PGQ. Esta brecha motiva la pregunta central de la presente investigación: *¿cómo mitigar de forma sistemática las alucinaciones estructurales y semánticas en la generación* text-to-query *sobre esquemas híbridos SQL/PGQ, de modo que las consultas resultantes sean correctas, verificables y operables en contextos empresariales?*

1.3. Objetivos

Objetivo general: Proponer una arquitectura de razonamiento estructural basada en RI para la generación *text-to-query* orientada al estándar SQL/PGQ, que mitigue las alucinaciones de esquema y los errores semánticos en escenarios de consulta empresarial.

1.3.1. Objetivos Específicos

- Caracterizar los tipos de alucinación en la generación *text-to-query* mediante una taxonomía de errores aplicable a consultas relacionales e híbridas, identificando sus causas computacionales.
- Diseñar una RI unificada para consultas híbridas SQL/PGQ que integre componentes relacionales y de patrón de grafo con reglas de tipado y compilación determinista.
- Implementar un *pipeline* de razonamiento estructural que incorpore anclaje al esquema, generación restringida por gramática y un bucle de verificación estática y dinámica.
- Evaluar experimentalmente la arquitectura propuesta frente a métodos del estado del arte mediante métricas de corrección, tasa de alucinación, robustez y eficiencia sobre *benchmarks* establecidos.

1.4. Organización de la tesis

El resto de este documento está organizado de la siguiente manera:

- El capítulo 2 presenta los fundamentos conceptuales y técnicos: LLMs y alucinaciones, *text-to-SQL*, el estándar SQL/PGQ, y las técnicas de RI, decodificación restringida y verificación.
- El capítulo line:60 **??** revisa las arquitecturas y enfoques actuales en *text-to-query* (pipelines por etapas, agentes, *constrained decoding*, detección y mitigación de alucinaciones) y posiciona el problema en el contexto empresarial y del estándar híbrido.
- El capítulo 4 describe la propuesta: la RI tipada para SQL/PGQ, el compilador de RI a SQL/PGQ, el pipeline de razonamiento estructural y el bucle de verificación (estático y por ejecución).
- El capítulo 5 detalla el diseño experimental: datasets/benchmarks, construcción de corpus híbrido para SQL/PGQ, métricas de alucinación y corrección, baselines y *ablations*.
- El capítulo 6 presenta y analiza los resultados obtenidos, incluyendo robustez a *schema drift*, seguridad/eficiencia, y evaluación de componentes, validando los resultados con datasets relevantes.

- El capítulo 7 concluye la tesis, resume aportes, limitaciones y líneas de trabajo futuro.

1.5. Cronograma

Tabla 1.1: Cronograma de actividades de la tesis.

		N.°	Actividad	Detalles	Semana	Fecha	%
Semestre 2026-1	Parcial	1	Alineamiento inicial del trabajo	Revisión del curso, entregables, estructura de la tesis y ajuste del tema con enfoque en SQL/PGQ + representación intermedia.	Sem. 1	11/03 – 14/03	2
		2	Definición final de tema y asesor	Alcance, contribución central y validación con asesora. Primera revisión de tabla survey.	Sem. 2	14/03 – 21/03	5
		3	Formalización y delimitación	Compromiso de asesoría, planteamiento del problema, objetivo general y específicos.	Sem. 3	21/03 – 28/03	10
		4	Consolidación del survey	Depuración del estado del arte, clasificación de trabajos, taxonomía y bibliografía base.	Sem. 4	28/03 – 01/04	15
		5	Redacción de Introducción v1	Motivación, contexto, problema, objetivos, organización y cronograma.	Sem. 5	01/04 – 08/04	18
		6	Cierre de Introducción	Ajustes con feedback y versión entregable del capítulo Introducción.	Sem. 5	08/04 – 11/04	20
		7	Redacción de Marco Teórico	Fundamentos teóricos, conceptos clave y modelos explicativos de la investigación.	Sem. 6	11/04 – 14/04	28
		8	Redacción de Estado del Arte	Literatura reciente, trabajos previos, vacíos de investigación y posicionamiento.	Sem. 6	14/04 – 18/04	35
		9	Capítulo Propuesta v1	Arquitectura, pipeline, representación intermedia, componentes y flujo de verificación.	Sem. 7	18/04 – 25/04	45
		10	Capítulo Propuesta v2	Protocolo experimental, datasets, métricas, baselines y criterios de evaluación.	Sem. 8	26/04 – 02/05	50
		11	Integración del Plan de Tesis	Consolidación de capítulos, coherencia interna, flujo narrativo y bibliografía final.	Sem. 9	03/05 – 09/05	55
		12	Plan de Tesis listo para envío	Cierre de documento, diapositivas y ensayo previo a la exposición.	Sem. 10	11/05 – 16/05	55
	Final	13	Implementación base experimental	Repositorio, estructura del prototipo, casos de prueba y baseline(s).	Sem. 11	17/05 – 23/05	62
		14	Implementación arquitectura v1	Flujo NL → RI → consulta estructurada/verificable.	Sem. 12	24/05 – 30/05	68
		15	Integración end-to-end y validación	Integración de componentes, validaciones estructurales y depuración.	Sem. 13	31/05 – 06/06	73
		16	Experimentación preliminar	Pruebas comparativas, recopilación de resultados y análisis preliminar.	Sem. 14	07/06 – 13/06	77
		17	Resultados al 80 y Avances	Prototipo funcional, resultados parciales sólidos y figuras iniciales.	Sem. 15	14/06 – 20/06	80

Continúa en la siguiente página...

		N.°	Actividad	Detalles	Semana	Fecha	%
		18	Cierre del documento de Avances	Documento completo, conclusiones preliminares y preparación de diapositivas.	Sem. 16	21/06 – 27/06	80
		19	Envío y exposición de Avances	Presentación final del semestre con resultados parciales consolidados.	Sem. 17	28/06 – 04/07	80
Int.		20	Revisión y fortalecimiento	Análisis de trabajos complementarios y refinamiento de la arquitectura propuesta.		10/07 – 10/08	82
Semestre 2026-2	Parcial	21	Proyecto de Investigación	Propuesta formal: impacto, viabilidad y posible aplicación institucional.	Sem. 1–2	12/08 – 22/08	84
		22	Ampliación experimental	Nuevos escenarios, ablations, análisis de errores y validación empresarial.	Sem. 3–5	23/08 – 15/09	87
		23	Resultados finales y discusión	Comparación con baselines, análisis crítico y discusión de aportes.	Sem. 6–8	16/09 – 06/10	90
		24	Preparación y Exposición Parcial	Presentación con metodología, resultados preliminares y validación ante jurado.	Sem. 9	07/10 – 13/10	91
	Final	25	Revisión y ajustes post-parcial	Correcciones metodológicas y refinamiento con feedback del jurado.	Sem. 10	14/10 – 20/10	92
		26	Síntesis de resultados y divulgación	Hallazgos en formato de reporte técnico o borrador de artículo.	Sem. 11	21/10 – 27/10	93
		27	Redacción final de tesis	Cierre de capítulos, conclusiones, trabajo futuro y referencias.	Sem. 12–14	28/10 – 15/11	96
		28	Correcciones con asesora	Ajustes de fondo y forma, normalización y aprobación final.	Sem. 15–16	16/11 – 30/11	98
		29	Preparación y Exposición Final	Diapositivas, ensayo de tiempos y sustentación final ante jurado.	Sem. 17–18	01/12 – 12/12	100

Capítulo 2

Fundamentos de Generación Text-to-Query y Consultas Híbridas

Este capítulo presenta los conceptos necesarios para comprender la propuesta del presente trabajo la cual será desarrollada en el Capítulo 4. La exposición avanza de manera progresiva, desde los fundamentos de los modelos de lenguaje hasta los mecanismos que permiten controlar y verificar las consultas generadas.

En primer lugar, se describen los LLMs, su forma de generar texto y los tipos de alucinaciones que pueden aparecer en este proceso (Sección 2.1). Luego, se revisan los modelos de datos relacional y de grafos de propiedad, así como su integración mediante el estándar SQL/PGQ (Sección 2.2) y se introduce el problema de *semantic parsing* y, dentro de este, la tarea de *text-to-query* (Sección 2.3).

Posteriormente, se explica el papel de una representación intermedia tipada como puente entre la interpretación realizada por el modelo y la generación final de una consulta ejecutable (Sección 2.4) así como se presentan las gramáticas formales y la decodificación restringida como mecanismos para guiar la salida del LLM hacia estructuras válidas (Sección 2.5). Finalmente, se revisan las formas de verificación estática y dinámica que permiten evaluar la corrección de la consulta generada (Sección 2.6).

El capítulo concluye con una síntesis (Sección line:10 **??**) que relaciona estos elementos y muestra cómo sirven de base para el análisis del estado del arte y para la propuesta de esta investigación.

2.1. Modelos de lenguaje y generación estructurada

Los LLMs, o modelos extensos de lenguaje, son modelos basados en arquitecturas *transformer* [Vaswani et al., 2017] y entrenados sobre grandes volúmenes de texto. En los últimos años, estos modelos se han convertido en una base importante para la investigación en tareas de *text-to-query*, esto debido a su capacidad para interpretar preguntas en lenguaje natural y generar respuestas en formatos estructurados.

Para el desarrollo de esta tesis, su relevancia se entiende a partir de tres aspectos principales: En primer lugar, los LLMs pueden adaptarse a esquemas que no fueron vistos durante el entrenamiento, especialmente mediante aprendizaje en contexto [Brown et al., 2020]. Esto permite aplicar el modelo a nuevos dominios sin requerir necesariamente un proceso completo de reentrenamiento. En segundo lugar, el uso de vocabularios basados en *subwords* les permite representar identificadores propios de un esquema, como nombres de tablas, columnas o relaciones, incluso cuando estos no aparecen literalmente en sus datos de entrenamiento. En tercer lugar, la disponibilidad de modelos preentrenados de propósito general permite distintas formas de adaptación, ya sea mediante *fine-tuning*, diseño del *prompt* o una combinación de ambos enfoques.

Sin embargo, esta capacidad de generación no resuelve por sí sola el problema de *text-to-query*, ya que los LLMs generan texto de forma autorregresiva, es decir, producen una secuencia de *tokens* prediciendo cada nuevo elemento a partir de los anteriores. Este proceso está guiado por patrones estadísticos aprendidos durante el entrenamiento, pero no incorpora de manera nativa las reglas gramaticales del lenguaje de consulta ni las restricciones del esquema sobre el cual se debe operar, dando como resultado que el modelo pueda producir consultas que parezcan correctas desde el punto de vista del lenguaje natural, pero que contienen errores estructurales, referencias inexistentes o interpretaciones equivocadas del esquema. Estos errores forman parte de los fenómenos que la literatura denomina *alucinaciones* [Ji et al., 2023].

2.1.1. Arquitectura transformer y decodificación autorregresiva

La arquitectura *transformer* [Vaswani et al., 2017] permite procesar una secuencia de entrada $x = (x_1, \ldots, x_n)$ y representar cada *token* mediante vectores contextualizados. Para ello utiliza mecanismos de atención, los cuales permiten que el modelo relacione distintos elementos de la secuencia, incluso cuando se encuentran alejados entre sí. A diferencia de arquitecturas recurrentes, este procesamiento no depende de recorrer la secuencia paso a paso, sino de comparar los *tokens* entre sí mediante capas de atención.En tareas de *text-to-query*, los LLMs suelen emplearse como modelos generativos autorregresivos. Esto significa que generan la salida $y = (y_1, \ldots, y_m)$ un *token* a la vez, donde cada nuevo *token* depende de los generados previamente y de la entrada original. Formalmente, esta generación puede expresarse como:

$$P(y \mid x) \;=\; \prod_{t=1}^{m} P(y_t \mid y_{<t},\, x),$$

donde $P(y_t \mid y_{<t}, x)$ representa la probabilidad del siguiente *token* dado el prefijo ya generado $y_{<t}$ y la entrada x. En la práctica, esta probabilidad se obtiene mediante una capa *softmax* sobre el vocabulario del modelo. A partir de esa distribución, el siguiente *token* puede seleccionarse mediante estrategias como la maximización local o el muestreo.Esta forma de generación tiene dos consecuencias importantes para *text-to-query*. En primer lugar, el modelo decide cada paso en función de la probabilidad del siguiente *token*, pero no garantiza por sí mismo que la consulta completa sea válida. Por ejemplo, una elección localmente probable puede introducir el nombre de una columna inexistente o una relación incorrecta entre tablas. En segundo lugar, el espacio de salida del modelo está definido por *tokens*, no por las reglas formales del lenguaje de consulta. Por ello, aunque el modelo

pueda producir una secuencia que se parezca a una consulta válida, no existe una garantía nativa de que dicha secuencia respete la gramática del lenguaje ni las restricciones del esquema objetivo.

2.1.2. Prompting y aprendizaje en contexto

Sobre esta base de generación, la aplicación a tareas específicas admite dos regímenes complementarios. El *fine-tuning* supervisado modifica los parámetros del modelo mediante pares (pregunta, consulta) con el fin de sesgar $P(y \mid x)$ hacia el lenguaje objetivo. El *aprendizaje en contexto* [Brown et al., 2020], por su parte, mantiene los parámetros congelados y condiciona la generación en un *prompt* que incluye instrucciones y, opcionalmente, ejemplos de la tarea; en este régimen, la distribución efectiva de generación depende de la composición del *prompt*, no de una actualización del modelo.

Ambos regímenes comparten una limitación estructural: la salida es, en último término, el resultado de un muestreo sobre una distribución que desconoce las restricciones formales del lenguaje de consulta y del esquema. Por ello, el estudio de la fidelidad estructural de la salida requiere mecanismos externos al propio LLM.

2.2. Modelos de datos y lenguajes de consulta

Las consultas estudiadas en esta investigación se formulan sobre dos modelos de datos: el modelo relacional y el modelo de grafos de propiedad. Aunque ambos pueden representar información relacionada, lo hacen mediante estructuras distintas y con lenguajes de consulta diferentes. Por ello, esta sección presenta primero cada modelo por separado y luego explica su integración mediante el estándar SQL/PGQ.

2.2.1. Modelo relacional y álgebra relacional

El modelo relacional [Codd, 1970] representa los datos mediante *relaciones*: conjuntos de tuplas de aridad fija, donde cada posición se asocia a un atributo con dominio de valores. Formalmente, una relación R sobre el esquema $(A_1, \ldots, A_k)$ con dominios $(D_1, \ldots, D_k)$ se define como

$$R \subseteq D_1 \times \cdots \times D_k.$$

Un *esquema relacional* $\mathcal{S}_{\text{rel}}$ es un conjunto finito de tales relaciones, junto con restricciones de integridad: claves primarias, claves foráneas, unicidad y restricciones de dominio.

Sobre este modelo opera el álgebra relacional, cuyos operadores básicos $\{\sigma, \pi, \bowtie, \cup, -, \rho\}$, que corresponden respectivamente a selección, proyección, unión natural, unión, diferencia y renombramiento, son cerrados sobre el conjunto de relaciones y componen un lenguaje declarativo con expresividad equivalente al cálculo relacional de primer orden con dominio acotado (*safe range calculus*). Esta equivalencia es metodológicamente relevante: propiedades como la existencia de una columna referenciada o la consistencia de una unión

respecto a las claves foráneas admiten formalización como predicados decidibles sobre el álgebra, lo que habilita la verificación estática en tiempo lineal sobre el tamaño de la consulta.

2.2.2. SQL como lenguaje declarativo

SQL (*Structured Query Language*), estandarizado en la serie ISO/IEC 9075, es el lenguaje declarativo dominante para consultar bases de datos relacionales. Una consulta en forma `SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY` se corresponde, con matices sintácticos, con una expresión del álgebra relacional extendida con agregaciones y ordenamiento. La distinción entre la consulta declarativa y el plan físico ejecutado por el motor es relevante para esta tesis: el objetivo de la generación es producir una expresión declarativa bien formada y grounded al esquema; la optimización y ejecución son responsabilidad del RDBMS.

2.2.3. Grafos de propiedad y pattern matching

Un *grafo de propiedad* [Angles et al., 2017] se define como una estructura

$$G \;=\; (V,\, E,\, \lambda,\, \mathrm{src},\, \mathrm{tgt},\, \mathrm{prop}),$$

donde V es un conjunto finito de vértices, E un conjunto finito de aristas, $\lambda : V \cup E \to 2^{\Sigma_L}$ asigna a cada elemento un conjunto de etiquetas de un alfabeto Σ_L, $\mathrm{src}, \mathrm{tgt} : E \to V$ determinan los extremos de cada arista dirigida, y $\mathrm{prop} : (V \cup E) \times K \rightharpoonup D$ asocia a cada elemento y cada clave $k \in K$ un valor parcial en un dominio D.

El lenguaje de consulta natural sobre esta estructura es el *pattern matching*: una consulta se expresa como un patrón P, esto es, un grafo con variables en lugar de identificadores, y el resultado es el conjunto de homomorfismos de P en G compatibles con las etiquetas y con las restricciones declaradas sobre las propiedades. Como ilustración mínima, sobre un grafo en el que los vértices llevan la etiqueta `Person` y las aristas la etiqueta `KNOWS`, el patrón

$$(p_1{:}\texttt{Person})\ \texttt{KNOWS}\ (p_2{:}\texttt{Person})\ \texttt{KNOWS}\ (p_3{:}\texttt{Person})$$

denota la consulta "personas conectadas por dos saltos de `KNOWS`"; cada homomorfismo de este patrón en G instancia las variables p_1, p_2, p_3 en vértices concretos y se proyecta como una tupla del resultado.

Una clase particularmente estudiada son las *regular path queries* (**RPQ!**s (**RPQ!**s)), que especifican caminos mediante expresiones regulares sobre el alfabeto Σ_L; generalizan las consultas de vecindad y admiten evaluación en tiempo polinomial mediante construcciones basadas en autómatas [Angles et al., 2017]. Esta formalización permite, a efectos de verificación, formular reglas de tipado análogas a las del modelo relacional: toda referencia en una consulta sobre grafos debe corresponder a una etiqueta, un tipo o una propiedad declarada en la especificación de G.

2.2.4. El estándar SQL/PGQ como integración

El estándar ISO/IEC 9075-16:2023 [ISO/IEC JTC 1/SC 32, 2023] introduce SQL/PGQ como extensión de SQL para consultar grafos de propiedad definidos sobre tablas relacionales existentes. Su construcción fundamental es la expresión `GRAPH_TABLE(graph MATCH pattern COLUMNS (...))`, que produce una relación derivada a partir de un patrón de grafo y admite composición directa con las cláusulas relacionales clásicas. La integración es conceptualmente significativa por dos razones. Primera, unifica dos modelos de datos formalmente distintos bajo un único lenguaje, con lo cual el problema de la generación *text-to-query* debe abarcar, simultáneamente, el álgebra relacional y el *pattern matching* sobre grafos. Segunda, introduce una nueva categoría de consistencia estructural: la coherencia entre los componentes relacional y de grafo de una misma consulta, que no se reduce a la validez de cada componente por separado.

2.3. Semantic parsing y generación *text-to-query*

2.3.1. *Semantic parsing*: definición y marco general

El *semantic parsing* [Zelle and Mooney, 1996] es la tarea de transformar un enunciado en lenguaje natural u en una representación formal z que pueda ser ejecutada sobre un entorno $\mathcal{E}$. Este entorno puede ser, por ejemplo, una base de datos, un grafo de conocimiento o incluso un sistema físico. El objetivo es que la ejecución de dicha representación formal, denotada como $z_{\mathcal{E}}$, produzca la respuesta esperada para la pregunta original.

La representación formal z puede adoptar distintas formas, dependiendo del dominio y del tipo de sistema. Puede tratarse de expresiones en λ-cálculo, lenguajes específicos de dominio, consultas SQL, consultas sobre grafos como *Cypher*, entre otros. En términos generales, el problema requiere relacionar tres componentes: el enunciado en lenguaje natural, el entorno sobre el cual se ejecutará la consulta y el mecanismo que traduce el primero en una representación formal válida.

2.3.2. *Text-to-SQL* como caso particular y extensión a *text-to-query*

La tarea *text-to-SQL* es un caso particular de *semantic parsing*. En este caso, la representación formal pertenece al lenguaje SQL y el entorno de ejecución es una base de datos relacional con un esquema $\mathcal{S}_{\text{rel}}$. La entrada del sistema puede expresarse como el par $(u, \mathcal{S}_{\text{rel}})$, donde u es la pregunta en lenguaje natural y $\mathcal{S}_{\text{rel}}$ es el esquema de la base de datos. La salida esperada es una consulta q que, al ejecutarse sobre la base de datos, produzca la respuesta correcta.

La presente tesis aborda una extensión de este problema hacia *text-to-query* sobre SQL/PGQ. En este escenario, la consulta no se limita únicamente a operaciones relacionales, sino que también puede incluir patrones de grafo y combinaciones entre ambos modelos. Por ello, la entrada se define como un par $(u, \mathcal{S})$, donde $\mathcal{S}$ representa un esquema

dual:

$$\mathcal{S} \;=\; (\mathcal{S}_{\text{rel}},\, \mathcal{S}_{\text{grafo}}).$$

Aquí, $\mathcal{S}_{\text{rel}}$ corresponde al componente relacional, mientras que $\mathcal{S}_{\text{grafo}}$ describe el componente de grafo de propiedad. La salida esperada es una consulta capaz de combinar operadores SQL tradicionales con patrones de grafo definidos bajo SQL/PGQ.

2.3.3. *Schema linking* y *grounding* al esquema

Una subtarea central en los sistemas *text-to-query* es el *schema linking*. Esta consiste en identificar qué elementos del esquema son mencionados, directa o indirectamente, en la pregunta del usuario. Dichos elementos pueden ser tablas, columnas, relaciones, etiquetas de vértices, tipos de aristas o propiedades del grafo.

Formalmente, esta correspondencia puede representarse mediante una función:

$$\ell : \text{menciones}(u) \to 2^{\mathcal{S}},$$

donde cada mención del enunciado u se asocia con un subconjunto de elementos del esquema $\mathcal{S}$ que podrían corresponder a dicha mención.

El *grounding* al esquema se refiere a la propiedad de que toda referencia utilizada en la consulta generada debe estar vinculada a un elemento existente y relevante del esquema. En otras palabras, la consulta no debería introducir tablas, columnas, etiquetas, aristas o propiedades que no estén definidas en $\mathcal{S}$ o que no hayan sido identificadas como pertinentes para la pregunta.

Cuando este proceso falla, aparecen alucinaciones referenciales, como se describió en la Sección line:107 **??**. Por esta razón, un sistema orientado a reducir este tipo de errores debe incorporar mecanismos explícitos de *grounding*. En la práctica, esto suele realizarse mediante una fase previa a la generación, en la que se seleccionan y organizan los elementos del esquema que serán considerados por el modelo.

2.4. Representaciones intermedias

2.4.1. Noción formal de representación intermedia

Una *representación intermedia* (RI) es un lenguaje formal $\mathcal{L}_{\text{RI}}$ dotado de una gramática $\mathcal{G}_{\text{RI}}$, un sistema de tipos $\mathcal{T}_{\text{RI}}$ y una función de compilación

$$\kappa : \mathcal{L}_{\text{RI}} \to \mathcal{L}_{\text{objetivo}}$$

hacia el lenguaje de consulta efectivo, que en esta tesis es SQL/PGQ. Su rol en un pipeline de generación es mediar entre la salida del componente estadístico y la consulta ejecutable: el modelo produce, o es guiado a producir, instancias bien tipadas de $\mathcal{L}_{\mathrm{RI}}$; el compilador κ genera determinísticamente la consulta final.

2.4.2. Propiedades deseables

Una RI adecuada para *text-to-query* satisface, idealmente, las siguientes propiedades:

Abstracción sintáctica. $\mathcal{L}_{\mathrm{RI}}$ suprime decisiones sintácticas irrelevantes del lenguaje objetivo, tales como los alias, el orden de cláusulas o la elección entre `JOIN` y subconsulta. Dos consultas objetivo semánticamente equivalentes admiten, idealmente, una única representación en $\mathcal{L}_{\mathrm{RI}}$.

Tipado decidible. La corrección estructural de una instancia se decide mediante reglas locales, sin ejecutar la consulta. Cada nodo declara los tipos de sus operandos, y las reglas de tipado se verifican composicionalmente.

Compilación determinista. κ es una función total sobre instancias bien tipadas y produce la misma salida para instancias estructuralmente idénticas. Esto aísla la indeterminación del modelo en el espacio de $\mathcal{L}_{\mathrm{RI}}$.

Expresividad adecuada. $\mathcal{L}_{\mathrm{RI}}$ cubre las construcciones del lenguaje objetivo relevantes para la tarea. En SQL/PGQ, esto exige soporte para operadores relacionales, patrones de grafo y su composición híbrida.

2.4.3. Separación entre generación estadística y compilación determinista

La introducción de una RI permite separar, en el pipeline, la generación estadística, que produce hipótesis de interpretación, de la compilación determinista, que produce la consulta ejecutable. Esta separación es metodológicamente relevante: las alucinaciones del modelo quedan confinadas al espacio de $\mathcal{L}_{\mathrm{RI}}$ y se detectan mediante verificación de tipado antes de que alcancen el motor de ejecución. La propuesta del Capítulo 4 instancia esta idea con una RI específica para consultas híbridas SQL/PGQ.

2.5. Gramáticas formales y decodificación restringida

2.5.1. Gramáticas formales y jerarquía de Chomsky

Una gramática formal [Chomsky, 1956] se define como una tupla $\mathcal{G} = (N, \Sigma, P, S)$, donde N es un conjunto finito de símbolos no terminales, Σ un alfabeto de terminales, P

un conjunto de producciones y $S \in N$ el símbolo inicial. Los lenguajes de consulta de interés para esta tesis pertenecen, en su núcleo sintáctico, a la clase libre de contexto: tanto SQL como SQL/PGQ admiten descripciones mediante gramáticas libres de contexto. La correspondencia entre un identificador y su declaración en el esquema, en cambio, no es decidible por la gramática y requiere un análisis adicional sobre el contexto de tipos. Esta separación entre validez gramatical y validez referencial motiva la combinación de decodificación restringida por gramática con verificación por tipos, descritas a continuación.

La teoría del parsing [Aho et al., 2006] distingue entre estrategias *top-down* (análisis descendente, típicamente LL) y *bottom-up* (análisis ascendente, típicamente LR). Para el propósito de esta tesis, la propiedad relevante no es el tipo específico de parser, sino la posibilidad de realizar *parsing incremental*: validar un prefijo parcial de la secuencia de salida contra la gramática mientras la generación está en curso.

2.5.2. Decodificación restringida como acoplamiento generación–parser

La decodificación restringida acopla el proceso autorregresivo de un LLM a un parser incremental de $\mathcal{G}$. En cada paso t, el parser mantiene el conjunto $C_t \subseteq \Sigma$ de *tokens* que extienden el prefijo $y_{<t}$ sin violar la gramática, y la distribución de salida del modelo se restringe a C_t:

$$\tilde{P}(y_t \mid y_{<t},\, x) \;\propto\; P(y_t \mid y_{<t},\, x) \cdot 1[\, y_t \in C_t \,].$$

Bajo esta restricción, toda secuencia generada es, por construcción, sintácticamente válida respecto a $\mathcal{G}$. La propiedad es estructural, no estadística: no depende de la calidad del modelo. Sin embargo, la decodificación restringida no garantiza *grounding*: una secuencia puede ser sintácticamente válida y, al mismo tiempo, referenciar identificadores ausentes del esquema, lo que motiva la introducción de mecanismos de verificación complementarios.

2.5.3. Sistemas de tipos y reglas locales de validación

Un sistema de tipos [Pierce, 2002] añade a una gramática un conjunto de reglas de inferencia que determinan cuándo una expresión está bien tipada. Las reglas se expresan como juicios de la forma $\Gamma \vdash e : \tau$, que se leen: "bajo el contexto Γ, la expresión e tiene tipo τ". Un ejemplo canónico es la regla de aplicación:

$$\frac{\Gamma \vdash f : \tau_1 \to \tau_2 \qquad \Gamma \vdash a : \tau_1}{\Gamma \vdash f(a) : \tau_2}.$$

Aplicado a una RI para *text-to-query*, el contexto Γ codifica el esquema dual $(\mathcal{S}_{\text{rel}}, \mathcal{S}_{\text{grafo}})$, y las reglas de tipado expresan propiedades como: toda referencia a una columna debe corresponder a una tabla del contexto; toda proyección de propiedad debe apuntar a un tipo de vértice o arista declarado; la unión de dos bloques relacionales exige esquemas compatibles. La decidibilidad local de las reglas las hace adecuadas para verificación estática en tiempo lineal sobre el tamaño de la RI.

2.5.4. Alcance y límites formales

Las técnicas de decodificación restringida y verificación por tipos presentan un alcance preciso y límites igualmente precisos. Garantizan la validez sintáctica y la consistencia referencial, pero no deciden, en general, la fidelidad semántica de la consulta respecto a la intención del usuario, ya que esta última no admite especificación formal previa. La estrategia global de una arquitectura robusta consiste, entonces, en combinar estas garantías formales con mecanismos de verificación dinámica, que se describen en la sección siguiente.

2.6. Verificación de consultas generadas

2.6.1. Verificación estática

La verificación estática designa el análisis de una consulta, o de su RI, sin ejecutarla, mediante la aplicación de reglas decidibles sobre su estructura y sobre el esquema. En el contexto de *text-to-query*, los chequeos estáticos relevantes incluyen: existencia de tablas, columnas, etiquetas y propiedades referenciadas; consistencia de tipos en operadores y condiciones; satisfacibilidad trivial de restricciones, como detección de condiciones contradictorias evidentes; y, en consultas híbridas, coherencia entre los componentes relacional y de grafo.

Formalmente, la verificación estática puede modelarse como un conjunto de predicados $\Phi = \{\varphi_1, \ldots, \varphi_k\}$ sobre consultas y esquemas. Una consulta q se considera *estáticamente válida* respecto a un esquema $\mathcal{S}$ si se cumple $\bigwedge_{i=1}^{k} \varphi_i(q, \mathcal{S})$. La decidibilidad local de cada φ_i hace de la verificación estática un mecanismo de costo constante o lineal respecto al tamaño de q, y por ello resulta aplicable en un bucle de refinamiento sin comprometer la eficiencia global.

2.6.2. Verificación dinámica y oráculos parciales

La verificación dinámica ejecuta la consulta en un entorno controlado y analiza la respuesta del motor. El motor actúa como *oráculo parcial*: distingue consultas sintáctica y referencialmente válidas de aquellas que disparan errores de ejecución, pero no decide, en general, la corrección semántica de una consulta ejecutable. Las señales útiles en este régimen incluyen los errores tipados del motor, la cardinalidad anómala del resultado, entendida como un resultado vacío cuando se espera contenido o viceversa, la presencia inesperada de valores nulos y las desviaciones respecto a respuestas de referencia cuando estas existen.

2.6.3. Testing metamórfico

El *testing metamórfico* [Chen et al., 1998] extiende la verificación dinámica más allá de los oráculos explícitos. La idea central consiste en identificar relaciones entre entradas y salidas que deben preservarse bajo transformaciones conocidas, denominadas *relaciones metamórficas*. En *text-to-query*, relaciones como la invariancia del resultado bajo renombramiento de alias, la inclusión del resultado de una consulta con `LIMIT` en el de la misma consulta sin dicha cláusula, o la preservación de cardinalidades bajo reformulaciones semánticamente equivalentes, ofrecen un mecanismo para detectar inconsistencias sin conocer de antemano la respuesta correcta. La violación de una relación metamórfica evidencia un error; su satisfacción, sin embargo, no certifica corrección, sino que acota el espacio de hipótesis de error.

2.6.4. Bucles de refinamiento

Un bucle de refinamiento combina las verificaciones anteriores en un proceso iterativo: una consulta que falla una verificación produce una señal estructurada que se retroalimenta al generador para intentar producir una consulta corregida. El diseño del bucle está gobernado por dos decisiones. La primera es la política de retroalimentación, que determina qué información derivada del fallo se entrega al generador y en qué forma: mensaje crudo del motor, descriptor categórico o sugerencia estructurada. La segunda es el criterio de terminación, que limita el número de iteraciones y decide cuándo declarar la pregunta intrínsecamente ambigua o no resoluble bajo el contexto disponible.

Capítulo 3

Arquitecturas y Enfoques Actuales en Text-to-Query

El capítulo anterior presentó los fundamentos necesarios para comprender la generación *text-to-query*: los modelos de lenguaje, sus posibles alucinaciones, las representaciones intermedias, la decodificación restringida, los mecanismos de verificación y el estándar híbrido SQL/PGQ. A partir de esa base, este capítulo revisa los principales enfoques actuales en sistemas *text-to-query*.

La revisión no se plantea como una lista cronológica de trabajos, sino como un análisis organizado por dimensiones. Esta decisión responde a que la literatura reciente aborda el problema desde criterios diversos: algunas revisiones clasifican los sistemas según la arquitectura del modelo utilizado, otras según la etapa del *pipeline* que buscan mejorar, y otras según la estrategia de *prompting* empleada [Quamar et al., 2022]. Aunque estas clasificaciones son útiles, no siempre permiten comparar los trabajos bajo criterios comunes ni identificar con claridad las brechas de investigación.

Por ello, este capítulo adopta una taxonomía multidimensional para analizar la literatura de manera más ordenada. Esta taxonomía distingue entre el paradigma de aprendizaje, la arquitectura de orquestación, la estrategia de representación, los mecanismos de enriquecimiento contextual, el tipo de razonamiento utilizado y las propiedades evaluativas que cada enfoque busca optimizar.

Cada sección desarrolla una de estas dimensiones, presenta los enfoques más relevantes y cierra con observaciones críticas sobre sus alcances y limitaciones. Finalmente, el capítulo sintetiza los hallazgos principales y formula la brecha de investigación que motiva la propuesta desarrollada en esta tesis.

3.1. Marco taxonómico de análisis

Resolver el problema *text-to-query* en entornos empresariales obliga a considerar simultáneamente decisiones de aprendizaje, de orquestación arquitectónica, de representación, de enriquecimiento contextual, de estrategia de razonamiento y de criterios de

evaluación. Estas decisiones operan en planos distintos del sistema y, en la práctica, se han combinado de formas muy variadas en la literatura. Para categorizar correctamente esta diversidad es importante considerar que:

1. La generación *text-to-query* es una tarea de traducción entre lenguaje natural y un lenguaje de consulta formal sujeto a un esquema específico.
2. Los sistemas pueden operar con un LLM cuyos parámetros se mantienen fijos, con un LLM ajustado a la tarea, o mediante combinaciones de ambas estrategias.
3. La generación puede producirse en una sola invocación del modelo o distribuirse entre módulos especializados que cooperan en un *pipeline* o en un esquema multi-agente.
4. La consulta final puede generarse directamente sobre el lenguaje objetivo o pasar primero por una representación estructurada intermedia que separe la interpretación semántica de la sintaxis del lenguaje destino.
5. El conocimiento contextual disponible para el modelo —esquema, contenido de la base de datos, ejemplos demostrativos, documentación de dominio— puede aprovecharse de formas distintas y con diferentes garantías de fidelidad.
6. El razonamiento que conduce de la pregunta a la consulta puede ser directo, descompuesto en pasos intermedios, generado en múltiples candidatos paralelos o iterado mediante retroalimentación.
7. Los criterios bajo los cuales un sistema se considera adecuado varían según el contexto: corrección semántica, eficiencia de ejecución, robustez ante variaciones del esquema, adaptabilidad a nuevos dominios, escalabilidad y nivel de interpretabilidad de la consulta generada.

Por lo tanto, en base a estas consideraciones, se desarrolló una taxonomía con seis dimensiones ortogonales para clasificar los sistemas *text-to-query*. Las dimensiones se diseñaron para aislar aspectos conceptualmente independientes, evitar la mezcla de niveles de abstracción presente en clasificaciones existentes y permitir el etiquetado múltiple cuando un trabajo combine varias categorías dentro de una misma dimensión. A continuación se detalla cada dimensión:

1. **Paradigma de aprendizaje:** cómo se ajusta, o no, el modelo base a la tarea *text-to-query*. Considera tres ramas:
   a) *Parameter-frozen*: los parámetros del modelo no se modifican; la adaptación ocurre en el *prompt*, ya sea sin ejemplos (*zero-shot*), con ejemplos fijos (*few-shot in-context learning*) o con ejemplos seleccionados dinámicamente mediante recuperación.
   b) *Parameter-tuned*: los parámetros se modifican mediante *fine-tuning* supervisado, preentrenamiento continuo, aprendizaje multi-tarea o aprendizaje por refuerzo.

   c) *Híbridos*: combinan componentes congelados con componentes ajustados, típicamente un generador adaptado y un seleccionador o reordenador con parámetros propios.

2. **Arquitectura del sistema:** cómo se organiza el cómputo entre invocaciones del modelo y módulos auxiliares. Se distinguen arquitecturas *monolíticas* (una sola invocación que produce la consulta), *pipelines modulares* (etapas secuenciales con responsabilidades separadas), *multiagente* (agentes con roles definidos que cooperan), *agentic-search* (exploración guiada del espacio de soluciones) y *generate-and-rerank* (generación de varios candidatos seguida de selección).

3. **Representación:** qué estructuras intermedias intervienen entre la entrada en lenguaje natural y la consulta final. Esta dimensión se subdivide en tres ejes complementarios:

   a) *Codificación de entrada*: cómo se representan la pregunta y el esquema antes de generar (texto plano, esquema serializado, grafos del esquema, vectores densos).
   b) *Representación intermedia*: si el sistema utiliza una RI explícita y de qué tipo, distinguiendo seis subcategorías —ausencia de RI, esquema serializado como RI implícita, lenguaje intermedio dedicado, plan estructurado, árbol abstracto y descomposición declarativa.
   c) *Decodificación*: cómo se restringe o no la generación de la consulta final (libre, con restricción gramatical, composicional o por compilación desde la RI).

4. **Mecanismos de enriquecimiento contextual:** qué información adicional se aporta al modelo para anclar su salida al esquema y al dominio. Considera cuatro mecanismos:

   a) *Schema linking*: asociación explícita entre menciones del texto y elementos del esquema.
   b) *Recuperación de contenido de la base de datos*: incorporación de valores reales de columnas para resolver ambigüedades léxicas.
   c) *Conocimiento externo*: integración de glosarios, documentación de dominio o restricciones de negocio.
   d) *Auto-refinamiento*: ciclos en los que el sistema observa errores en la consulta generada y la corrige.

5. **Paradigma de razonamiento:** cómo se estructura el proceso de inferencia que conduce a la consulta. Distingue razonamiento *directo*, en *cadena de pensamiento (chain-of-thought)*, *descompositivo*, de *múltiples caminos* y de *retroalimentación iterativa*.

6. **Propiedades evaluativas objetivo:** qué propiedades del sistema son priorizadas explícitamente por el trabajo, distinguiendo *corrección*, *eficiencia*, *robustez*, *adaptabilidad*, *escalabilidad* e *interpretabilidad*.

A partir de esta categorización, las secciones siguientes revisan la literatura reciente analizando, por dimensión, cómo se han posicionado los trabajos representativos. La revisión se concentra en propuestas publicadas mayoritariamente entre 2017 y 2026, abarcando

desde los primeros sistemas neuronales basados en codificadores especializados hasta las arquitecturas multiagente más recientes sustentadas en LLMs. Cada sección cierra con una observación crítica que identifica patrones, limitaciones y vacíos relevantes para la presente investigación.

3.2. Paradigma de Aprendizaje

Esta sección describe cómo los sistemas *text-to-query* se relacionan con su modelo base. Aunque la mayoría de los trabajos analizados se construyen sobre LLMs o sobre arquitecturas tipo *encoder-decoder* preentrenadas, se distinguen tres estrategias fundamentales: mantener los parámetros del modelo congelados, ajustarlos mediante entrenamiento adicional o combinar ambas estrategias en componentes distintos del sistema.

3.2.1. Aprendizaje con parámetros congelados

En esta categoría, la adaptación a la tarea ocurre exclusivamente a través del *prompt*, sin modificar los parámetros del modelo. La distinción relevante dentro de la categoría es cómo se construye dicho *prompt*.

(a) Zero-shot. Algunos trabajos prescinden por completo de ejemplos y se apoyan únicamente en una descripción del esquema y en la pregunta del usuario. C3-SQL [X. Dong and et al., 2023] es representativo de este enfoque: con un *prompt* cuidadosamente diseñado y sin ejemplos demostrativos, alcanza resultados competitivos sobre Spider apoyándose en la calidad del modelo subyacente. Alpha-SQL [B. Li and Luo, 2025], aun siendo un sistema agéntico, opera *zero-shot* en el sentido de que no requiere demostraciones específicas del dominio: explora el espacio de consultas mediante búsqueda guiada sobre prompts dinámicos. La principal limitación de esta sub-categoría es que su desempeño depende fuertemente del modelo base y se degrada notoriamente en esquemas complejos o atípicos.

(b) Few-shot in-context learning. La mayoría de los sistemas recientes basados en LLMs pertenece a esta sub-categoría. La consulta se genera condicionada a un conjunto fijo o seleccionado de ejemplos (pregunta, consulta) que ilustran el comportamiento esperado. DIN-SQL [Pourreza and Rafiei, 2023] fue un trabajo influyente en esta línea: descompone la tarea en sub-pasos y diseña *prompts* específicos para cada uno, con ejemplos curados manualmente. Sobre la misma base operan DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], SuperSQL [B. Li and et al., 2024], PET-SQL [Li and et al., 2024], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023], SteinerSQL [Xutao Mao, 2025], TS-SQL [Wenbo Xu, 2025], Wren AI [Chi, 2025], BLENDSQL [et al., 2025], TA-SQL [Qu et al., 2024], CoE-SQL [H. Zhang and et a PURPLE [T. Ren and et al., 2024], ODIS [Chang and Fosler-Lussier, 2023], SelECT-SQL [Z. Gu and et Gen-SQL [J. Shi and et al., 2025], SGU-SQL [Qinggang Zhang, 2025], CRED-SQL [Shaoming Duan, 202 y CRUSH4SQL [et al., 2023], entre otros. La diversidad de propuestas en esta sub-categoría

refleja la madurez del paradigma, pero también una saturación: las mejoras incrementales suelen depender de ingeniería de *prompts* más que de innovaciones arquitectónicas.

(c) In-context learning dinámico con recuperación. Una refinación relevante consiste en seleccionar los ejemplos que se incluyen en el *prompt* mediante recuperación, en función de la similitud con la pregunta entrante. OpenSearch-SQL [X. Xie and Guo, 2025] ilustra esta estrategia con un módulo dedicado de recuperación de ejemplos. El propio CRED-SQL [Shaoming Duan, 2025] contempla esta estrategia como extensión opcional. La principal ventaja sobre el *few-shot* estático es la adaptabilidad a dominios heterogéneos, aunque introduce dependencia de la calidad del corpus de ejemplos disponibles.

3.2.2. Aprendizaje con parámetros ajustados

En esta categoría se modifican los parámetros del modelo mediante entrenamiento adicional sobre datos específicos de la tarea. Es la familia históricamente más extendida en *text-to-SQL* previa al auge de los LLMs de propósito general.

(a) Fine-tuning supervisado. Trabajos como Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., RAT-SQL [B. Wang and et al., 2021], SmBoP [Rubin and Berant, 2021], RaSaP [J. Huang and et al., 2 SHiP [L. Dou and et al., 2022], RASAT [J. Qi and et al., 2022], PICARD [Scholak and et al., 2022], S²SQL [Hui and et al., 2022], Graphix-T5 [Li and et al., 2022], RESDSQL [H. Li and Chen, 2023], T5-3B+NatSQL [Gan and et al., 2023], SQLFormer [A. Bazaga and et al., 2023], CatSQL [H. Fu and et y DTS-SQL [Pourreza and Rafiei, 2024] ajustan modelos completos sobre conjuntos como Spider o WikiSQL. Las arquitecturas base varían desde redes específicas para *text-to-SQL* (Seq2SQL, BRIDGE, RAT-SQL, SmBoP) hasta variantes de T5 adaptadas (PICARD, RESDSQL, Graphix-T5). Los sistemas de esta sub-categoría históricamente han alcanzado las precisiones más altas en benchmarks controlados, aunque su costo de entrenamiento y su dependencia del dominio de entrenamiento limitan su transferencia directa a entornos empresariales.

(b) Preentrenamiento continuo. CodeS [H. Li and et al., 2024] representa una variante intermedia: parte de un modelo de código preentrenado de propósito general y aplica preentrenamiento continuo sobre un corpus masivo orientado a SQL antes del *fine-tuning* supervisado final. La estrategia mejora la transferencia a dominios desconocidos, pero requiere infraestructura considerable y un corpus de entrenamiento curado.

(c) Aprendizaje multi-tarea. Algunos trabajos entrenan el modelo sobre varias tareas relacionadas para mejorar la generalización. TKK [C. Gao and et al., 2022] aborda *text-to-SQL* junto con tareas auxiliares de comprensión de esquema. ROUTE [Y. Qin and et al., 2025] adopta el aprendizaje multi-tarea como principio organizador del *fine-tuning*, integrando tareas de generación, corrección y razonamiento sobre esquemas. PARSQL [Yaxun Dai, 2025] sigue una línea similar al combinar generación con selección entre candidatos. La motiva-

ción común es que las tareas auxiliares fuerzan al modelo a desarrollar representaciones internas más robustas del esquema y de la sintaxis SQL.

(d) Aprendizaje por refuerzo. STRuCT-LLM [Stoisser et al., 2025] incorpora aprendizaje por refuerzo para alinear la generación con señales de ejecución y con tareas de razonamiento sobre tablas y grafos simultáneamente. Seq2SQL [Victor Zhong, 2017], en una época mucho más temprana, ya empleaba aprendizaje por refuerzo basado en la recompensa de ejecución. La adopción del paradigma sigue siendo minoritaria pero ha resurgido con fuerza en propuestas recientes orientadas a unificar razonamiento estructurado con generación.

3.2.3. Estrategias híbridas

Una fracción creciente de los sistemas más recientes combina componentes con parámetros congelados y componentes ajustados. XiYan-SQL [Yingqi Gao, 2024] adopta una arquitectura multiagente en la que un generador puede ser un LLM ajustado de la familia XiYanSQL-QwenCoder mientras que módulos auxiliares operan *in-context*. CHASE-SQL [M. Pourreza and et al., 2025] combina generadores *in-context* con un seleccionador entrenado mediante *fine-tuning*. ZeroNL2SQL [Z. Gu and et al., 2024] distribuye la carga entre un modelo preentrenado pequeño y un LLM congelado. MetaSQL [Y. Fan and et al., 2024] aplica un esquema análogo en la fase de selección sobre múltiples candidatos. La motivación es pragmática: aprovechar la capacidad de generalización de los LLMs sin renunciar al control fino que provee el ajuste sobre los componentes críticos.

3.2.4. Observaciones críticas

La revisión de esta dimensión muestra un desplazamiento progresivo del *fine-tuning* de extremo a extremo hacia configuraciones híbridas y *in-context*. Esta tendencia es coherente con el aumento del costo computacional asociado al entrenamiento de LLMs de gran tamaño. Sin embargo, los sistemas puramente *in-context* acumulan dos limitaciones recurrentes: dependencia fuerte del modelo subyacente, lo que dificulta auditar el sistema desde el punto de vista de gobernanza, y ausencia de mecanismos formales para garantizar que la salida respete el esquema. La investigación de la presente tesis se posiciona como agnóstica al paradigma de aprendizaje subyacente: el énfasis se traslada a la representación intermedia y al bucle de verificación, de modo que la corrección estructural no descanse exclusivamente en la calidad estadística del generador.

3.3. Arquitectura del Sistema

Esta sección describe cómo se organiza el cómputo en los sistemas *text-to-query*. La distinción es independiente del paradigma de aprendizaje: un mismo paradigma puede dar

lugar a arquitecturas monolíticas, modulares, multiagente, agénticas o de generación con reordenamiento.

3.3.1. Arquitecturas monolíticas

En las arquitecturas monolíticas, una sola invocación del modelo produce la consulta completa. Es la configuración natural de los sistemas *end-to-end* entrenados para mapear pregunta a consulta directamente. Pertenecen a esta categoría Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], RAT-SQL [B. Wang and et al., 2021], SmBoP [Rubin and Berant, 2021] RaSaP [J. Huang and et al., 2021], RASAT [J. Qi and et al., 2022], PICARD [Scholak and et al., 2022], S²SQL [Hui and et al., 2022], Graphix-T5 [Li and et al., 2022], T5-3B+NatSQL [Gan and et al., 2023], SQLFormer [A. Bazaga and et al., 2023], CatSQL [H. Fu and et al., 2023], SHiP [L. Dou and et al., 202 CodeS [H. Li and et al., 2024], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023] ODIS [Chang and Fosler-Lussier, 2023], SGU-SQL [Qinggang Zhang, 2025] y STRuCT-LLM [Stoisser et al., 2025]. Aunque algunos de estos sistemas internamente realizan operaciones de codificación o decodificación sofisticadas, conceptualmente la generación de la consulta es una sola inferencia. La ventaja principal de esta organización es la simplicidad operativa; la desventaja es que cualquier error se propaga sin posibilidad de inspección intermedia ni de intervención modular.

3.3.2. Pipelines modulares

En esta categoría, la generación se distribuye entre etapas secuenciales con responsabilidades distintas: típicamente *schema linking*, generación, validación y refinamiento. DIN-SQL [Pourreza and Rafiei, 2023] popularizó una variante influyente del enfoque, descomponiendo el proceso en cuatro sub-tareas. La línea ha sido seguida por numerosos trabajos: RESDSQL [H. Li and Chen, 2023], DTS-SQL [Pourreza and Rafiei, 2024], PURPLE [T. Ren and et al., 2024], RSL-SQL [et al., 2024], DEA-SQL [Xie and et al., 2024], CoE-SQL [H. Zhang and et al., 2024], PET-SQL [Li and et al., 2024], SuperSQL [B. Li and et al., 2024] TA-SQL [Qu et al., 2024], FinSQL [Zhang and et al., 2024], TKK [C. Gao and et al., 2022], ZeroNL2SQL [Z. Gu and et al., 2024], SteinerSQL [Xutao Mao, 2025], Gen-SQL [J. Shi and et al., 2025] BLENDSQL [et al., 2025], CRED-SQL [Shaoming Duan, 2025], CRUSH4SQL [et al., 2023], SelECT-SQL [Z. Gu and et al., 2023], CHESS [S. Talaei and et al., 2024], C3-SQL [X. Dong and et al., y Wren AI [Chi, 2025]. Los *pipelines* modulares facilitan la inspección, el reemplazo de componentes y la integración con sistemas de verificación, lo que los hace especialmente atractivos para entornos empresariales. Su limitación principal es la rigidez del orden de ejecución y la posible amplificación de errores entre etapas.

3.3.3. Arquitecturas multiagente

Una variante reciente del enfoque modular dota a cada componente de autonomía e iniciativa, configurándolo como un agente que se comunica con otros mediante mensajes. MAC-SQL [B. Wang and et al., 2023] fue uno de los primeros trabajos en formalizar esta

organización para *text-to-SQL*, con agentes especializados en *schema linking*, descomposición y refinamiento. CHESS [S. Talaei and et al., 2024] extiende esta línea con cuatro agentes —recuperación de información, selección de esquema, generación y verificación— e incorpora un agente *Unit Tester* que actúa como verificador semántico. OpenSearch-SQL [X. Xie and Guo, 2025] y TS-SQL [Wenbo Xu, 2025] adoptan también una organización multiagente. XiYan-SQL [Yingqi Gao, 2024] combina varios agentes generadores con un agente seleccionador. La diferencia operativa entre un *pipeline* modular y un sistema multiagente es a menudo gradual; en la práctica, lo distintivo del enfoque multiagente es la capacidad de varios componentes de invocar al modelo de forma asíncrona o cíclica.

3.3.4. Arquitecturas agentic-search

Algunos trabajos abandonan la idea de un orden fijo y adoptan en su lugar una exploración guiada del espacio de soluciones. Alpha-SQL [B. Li and Luo, 2025] ilustra esta categoría: utiliza un modelo para proponer acciones (refinar el esquema, generar un fragmento, verificar una sub-consulta) y un mecanismo de búsqueda para decidir qué acción ejecutar a continuación. CHASE-SQL [M. Pourreza and et al., 2025] también incorpora elementos de exploración, aunque combinados con generación múltiple. La ventaja del enfoque es su flexibilidad ante consultas complejas; su desventaja, el costo computacional asociado a la búsqueda y la dificultad de garantizar terminación.

3.3.5. Arquitecturas de generación con reordenamiento

En esta categoría, el sistema produce varios candidatos de consulta y un módulo posterior selecciona el mejor. CHASE-SQL [M. Pourreza and et al., 2025], XiYan-SQL [Yingqi Gao, 2024] y MetaSQL [Y. Fan and et al., 2024] adoptan explícitamente esta estrategia. PARSQL [Yaxun Dai, 2025 también opera sobre un conjunto de candidatos y G^{3}R [**?** line:396] incorpora una etapa de reordenamiento basada en alineación. Trabajos clásicos como N-best List Rerankers [L. Zeng and Hakkani-Tur, 2022] sentaron las bases del enfoque. La motivación es robusta: si el generador produce un candidato correcto entre varios, un reordenador entrenado puede recuperarlo aún cuando no sea el más probable a priori. La debilidad es la dependencia de la diversidad real del conjunto generado.

3.3.6. Observaciones críticas

La distribución de los trabajos analizados por arquitectura muestra una migración progresiva desde sistemas monolíticos —propios de la era *pre-LLM*— hacia *pipelines* modulares, multiagente y agénticos. Esta evolución responde tanto a la creciente complejidad de las consultas objetivo como a la necesidad de inspeccionar y verificar la salida en entornos críticos. Sin embargo, una limitación común a las arquitecturas modulares y multiagente es que la verificación semántica, cuando existe, suele ejecutarse al final del proceso sobre la consulta ya generada, y rara vez se realiza sobre una representación intermedia que admita análisis estático. Esta brecha es uno de los puntos que la propuesta

de la presente tesis aborda explícitamente.

3.4. Estrategias de Representación

La forma en que un sistema representa la pregunta, el esquema y la consulta en formación condiciona qué tipos de errores puede detectar y qué tipos de control puede ejercer sobre la salida. Esta sección agrupa los trabajos según los tres ejes complementarios de la dimensión: codificación de la entrada, representación intermedia entre comprensión y generación, y estrategia de decodificación de la consulta final.

3.4.1. Codificación de la entrada

(a) Esquema serializado en lenguaje natural. La práctica dominante en sistemas basados en LLMs consiste en serializar el esquema como un fragmento de texto que se concatena con la pregunta. DIN-SQL [Pourreza and Rafiei, 2023], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], MAC-SQL [B. Wang and et al., 2023], CHESS [S. Talaei and et al., y la mayoría de los sistemas *in-context* recientes emplean esta codificación. Su ventaja es la simplicidad y la compatibilidad universal con LLMs; su desventaja es que se pierden las relaciones estructurales del esquema, salvo que se reintroduzcan explícitamente en el texto.

(b) Grafos del esquema. Una línea de trabajos modela explícitamente el esquema como un grafo, donde los nodos representan tablas y columnas y las aristas representan claves foráneas y otras relaciones. RAT-SQL [B. Wang and et al., 2021], S^2SQL [Hui and et al., 2022] y Graphix-T5 [Li and et al., 2022] incorporan capas de atención sensibles a la estructura del grafo. La motivación es facilitar el razonamiento sobre uniones complejas y rutas de *join* no triviales. El costo es una mayor complejidad arquitectónica y la necesidad de construir el grafo del esquema como paso de preprocesamiento.

(c) Esquema enriquecido (M-Schema y similares). XiYan-SQL [Yingqi Gao, 2024] introduce M-Schema, un formato que combina el esquema con metadatos y ejemplos de valores en una representación textual estructurada que el LLM consume directamente. Esta codificación intermedia entre texto plano y grafo busca preservar la simplicidad del primero sin perder la información estructural del segundo. Otros trabajos, como CHESS [S. Talaei and et al., 2024] y PET-SQL [Li and et al., 2024], adoptan formatos análogos en su etapa de preparación del *prompt*.

(d) Recuperación previa al modelo. En esquemas masivos, no es viable serializar el esquema completo en el *prompt*. CRUSH4SQL [et al., 2023] aborda esta restricción mediante recuperación: un módulo previo selecciona los elementos del esquema relevantes para la pregunta antes de invocar al modelo. TS-SQL [Wenbo Xu, 2025] y SuperSQL [B. Li and et al., 2024]

contemplan estrategias análogas. El compromiso es entre cobertura —no omitir elementos críticos— y compacidad —no saturar el contexto.

3.4.2. Representación intermedia

Esta sub-dimensión es central para la presente tesis. Se distinguen seis sub-categorías según el tipo de RI que cada sistema utiliza, si es que utiliza alguna.

(a) Sin RI explícita. La consulta se genera directamente como secuencia de tokens del lenguaje destino. Pertenecen a esta categoría Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], ODIS [Chang and Fosler-Lussier, 2023], SuperSQL [B. Li and et al., 2024] y la mayoría de los sistemas *in-context* basados en *prompting*. La ausencia de RI explícita simplifica la arquitectura, pero traslada toda la carga de corrección estructural al generador.

(b) Esquema como RI implícita. Algunos trabajos no introducen una RI dedicada, pero usan el esquema enriquecido como sustrato sobre el cual se realiza el razonamiento previo a la generación. Es el caso de RAT-SQL [B. Wang and et al., 2021] y, en parte, de XiYan-SQL [Yingqi Gao, 2024] con su M-Schema. La consulta sigue siendo SQL nativo, pero la representación del esquema actúa como puente estructurado.

(c) Lenguaje intermedio dedicado. La línea más explícita hacia una RI propiamente dicha la inicia IRNet con SemQL [Guo et al., 2019], un lenguaje intermedio que separa interpretación semántica de sintaxis SQL concreta. NatSQL [Gan and et al., 2023] sigue una motivación análoga: define un lenguaje SQL simplificado sobre el que se entrena el modelo, y un compilador trivial lo convierte a SQL ejecutable. CatSQL [H. Fu and et al., 2023] y ZeroNL2SQL [Z. Gu and et al., 2024] contemplan formas similares de representación intermedia. La ventaja estructural es que la RI restringe el espacio de errores posibles del modelo; la limitación es que la RI suele estar acoplada a un dialecto específico de SQL.

(d) Plan estructurado. Algunos sistemas representan la consulta como un plan de ejecución o como una secuencia de operaciones declarativas. SQLFormer [A. Bazaga and et al., 2023] sigue parcialmente esta línea al organizar la generación alrededor de operaciones primitivas. SteinerSQL [Xutao Mao, 2025] representa la consulta como un árbol sobre el grafo del esquema, lo que se aproxima a una representación plan-céntrica. La ventaja es la trazabilidad del razonamiento; la limitación es el costo de adoptar y mantener un formalismo propio.

(e) Árbol abstracto. SmBoP [Rubin and Berant, 2021] construye la consulta de forma composicional como un árbol abstracto sobre operadores relacionales, generando *bottom-up*. PICARD [Scholak and et al., 2022] no expone explícitamente un árbol como artefacto, pero su decodificación restringida opera implícitamente sobre la gramática del lenguaje,

lo que lo posiciona en una categoría afín. RaSaP [J. Huang and et al., 2021] y SQLFormer [A. Bazaga and et al., 2023] también incorporan estructura arbórea en distintos grados.

(f) Descomposición declarativa. DIN-SQL [Pourreza and Rafiei, 2023] introduce una forma específica de RI implícita: la pregunta se descompone en sub-preguntas y cada sub-pregunta produce una sub-consulta, que luego se ensambla. La descomposición no es una RI estructural, pero juega un papel análogo al distribuir el razonamiento. CHESS [S. Talaei and et al., 2024] y MAC-SQL [B. Wang and et al., 2023] adoptan estrategias similares.

3.4.3. Estrategias de decodificación

(a) Decodificación libre. El modelo genera la consulta como texto sin restricciones formales. Es la estrategia dominante en sistemas *in-context*: DIN-SQL [Pourreza and Rafiei, 2023], DAIL-SQL [D. Gao and et al., 2023], ACT-SQL [Zhang and et al., 2023], CHESS [S. Talaei and et al., 2 MAC-SQL [B. Wang and et al., 2023], OpenSearch-SQL [X. Xie and Guo, 2025], XiYan-SQL [Yingqi Gao, 2024], entre otros. Su ventaja es la flexibilidad; su desventaja, la posibilidad de generar consultas sintácticamente inválidas.

(b) Decodificación restringida por gramática. PICARD [Scholak and et al., 2022] establece el patrón de referencia de esta categoría: integra una gramática de SQL en el decodificador para descartar en tiempo real candidatos que violen el lenguaje. La técnica elimina por completo los errores sintácticos al costo de una decodificación más cara. PICARD inspira aplicaciones posteriores en otros componentes de *pipelines* modulares.

(c) Decodificación composicional. SmBoP [Rubin and Berant, 2021] genera la consulta de manera *bottom-up* sobre operadores relacionales, imponiendo estructura desde el inicio de la derivación. La estrategia es más compleja de implementar pero garantiza por construcción ciertos invariantes estructurales.

(d) Compilación desde la representación intermedia. Cuando el sistema utiliza una RI explícita, la decodificación final puede reducirse a la compilación determinista de la RI al lenguaje destino. IRNet [Guo et al., 2019] y NatSQL [Gan and et al., 2023] operan así: el modelo genera la RI y un componente determinista produce la consulta. Esta estrategia separa por completo el riesgo estadístico del riesgo formal.

3.4.4. Observaciones críticas

La revisión muestra que la mayoría de los sistemas recientes basados en LLMs ha priorizado simplificar la representación —esquema serializado, decodificación libre, sin RI explícita— delegando la corrección al generador. Esta decisión ha sido razonable mientras

los benchmarks dominantes contenían esquemas de complejidad moderada y consultas SQL relativamente regulares. Sin embargo, en escenarios empresariales y, especialmente, en consultas híbridas sobre el estándar SQL/PGQ, la ausencia de RI explícita amplifica la propagación de alucinaciones estructurales: no existe una capa intermedia donde verificar la consistencia de tipos, la validez de las uniones o la coherencia entre los componentes relacionales y de patrón de grafo. La investigación de esta tesis se posiciona deliberadamente en la sub-categoría de RI explícita con compilación determinista, ampliada para cubrir tanto SQL como PGQ y enriquecida con verificación estática sobre la propia RI.

3.5. Mecanismos de Enriquecimiento Contextual

Esta sección agrupa las técnicas que aportan información adicional al modelo más allá de la pregunta y el esquema básico. Los cuatro mecanismos no son excluyentes: muchos sistemas combinan varios.

3.5.1. Schema linking

El *schema linking* consiste en establecer correspondencias explícitas entre menciones del texto y elementos del esquema (tablas, columnas, valores). Esta operación es central en *text-to-SQL* y ha recibido tratamiento dedicado tanto como módulo independiente como integrado dentro de la arquitectura. RAT-SQL [B. Wang and et al., 2021] la incorpora mediante atención sensible a relaciones del esquema. RESDSQL [H. Li and Chen, 2023], DTS-SQL [Pourreza and Rafiei, 2024], DIN-SQL [Pourreza and Rafiei, 2023], DEA-SQL [Xie and et al., RSL-SQL [et al., 2024], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023], TA-SQL [Qu et al., 2024], PURPLE [T. Ren and et al., 2024], TS-SQL [Wenbo Xu, 2025] y CRUSH4SQL [et al., 2023], entre otros, dedican un módulo o etapa específica al *schema linking*. La calidad de esta etapa se correlaciona fuertemente con la corrección final del sistema, lo que explica la atención sostenida que ha recibido.

3.5.2. Recuperación de contenido de la base de datos

Algunos sistemas incorporan valores reales de las columnas para resolver ambigüedades léxicas que el esquema solo no permite resolver —por ejemplo, distinguir entre dos columnas con nombres similares pero contenido distinto. BRIDGE [Lin and et al., 2020] fue uno de los primeros en hacerlo de forma sistemática. Trabajos posteriores como CHESS [S. Talaei and et al., 2024], FinSQL [Zhang and et al., 2024], PET-SQL [Li and et al., 2024], CHASE-SQL [M. Pourreza and et al., 2025], CRED-SQL [Shaoming Duan, 2025] y SuperSQL [B. Li and et al., 2024] adoptan estrategias análogas, recuperando muestras de valores relevantes para la pregunta. La eficacia depende de la calidad del proceso de recuperación y plantea desafíos cuando los datos contienen información sensible.

3.5.3. Conocimiento externo y de dominio

En entornos empresariales, una fracción significativa de las consultas no puede resolverse correctamente sin conocimiento externo: glosarios de métricas, definiciones de dominio, restricciones de negocio. La línea de trabajos que aborda esta necesidad es comparativamente menor pero relevante. atscale2024sml [AtScale, Inc., 2024] ilustra el uso de un *semantic layer* corporativo. SAFENLIDB [Liu et al., 2025] aborda el componente de privacidad y seguridad de la integración con conocimiento externo. ROUTE [Y. Qin and et al., 2025] incorpora señales de robustez que implícitamente requieren conocimiento adicional sobre el dominio. La principal limitación de esta categoría es la falta de estandarización: cada trabajo define su propio formato para representar el conocimiento externo, lo que dificulta la comparación.

3.5.4. Auto-refinamiento

Una proporción creciente de sistemas incorpora ciclos en los que la consulta generada se ejecuta o se inspecciona, y los resultados o los errores se reintroducen al modelo para corrección. DIN-SQL [Pourreza and Rafiei, 2023] incluye una etapa de auto-corrección. MAC-SQL [B. Wang and et al., 2023], CHESS [S. Talaei and et al., 2024], XiYan-SQL [Yingqi Gao, 202 OpenSearch-SQL [X. Xie and Guo, 2025], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], PURPLE [T. Ren and et al., 2024] y CHASE-SQL [M. Pourreza and et al., 2025] adoptan variantes de este mecanismo. Los trabajos sobre detección de errores —ErrorLLM [Hong et al., 2026], SQLHD [Yang et al., 2025], AmbiSQL [Ding et al., 2026]— proveen señales que, en principio, podrían alimentar el bucle de auto-refinamiento. La eficacia del mecanismo depende crucialmente de la naturaleza de la retroalimentación: cuando es texto libre, tiende a inducir oscilaciones; cuando es estructurada, puede converger más rápidamente.

3.5.5. Observaciones críticas

El *schema linking* es el mecanismo más estudiado, mientras que la integración de conocimiento externo de dominio es claramente el menos desarrollado pese a su relevancia empresarial directa. El auto-refinamiento ha proliferado, pero la mayoría de las implementaciones operan sobre la consulta final como texto, lo que limita la precisión de la corrección posible y dificulta su análisis sistemático. Una arquitectura que opere sobre una RI tipada permite sustituir el bucle de auto-corrección sobre texto por retroalimentación estructurada que actúe sobre el nodo o la condición exacta donde se detectó el error.

3.6. Paradigma de Razonamiento

Esta sección agrupa los sistemas según cómo estructuran el proceso de inferencia que conduce de la pregunta a la consulta. El paradigma de razonamiento es independiente de la

arquitectura del sistema: una arquitectura monolítica puede emplear razonamiento directo o en cadena, y un *pipeline* modular puede operar de forma descompositiva o iterativa.

3.6.1. Razonamiento directo

El modelo produce la consulta sin pasos intermedios explícitos. Es la estrategia natural en sistemas *end-to-end*: Seq2SQL [Victor Zhong, 2017], BRIDGE [Lin and et al., 2020], RAT-SQL [B. Wang and et al., 2021], RASAT [J. Qi and et al., 2022], PICARD [Scholak and et al., 202 Graphix-T5 [Li and et al., 2022], RESDSQL [H. Li and Chen, 2023], T5-3B+NatSQL [Gan and et al., 2 y CodeS [H. Li and et al., 2024]. Estos sistemas confían el razonamiento al proceso interno del modelo y a las representaciones intermedias implícitas que aprende durante el entrenamiento.

3.6.2. Cadena de pensamiento

Aquí el modelo expone, antes de la consulta, una traza textual del razonamiento que conduce a ella. ACT-SQL [Zhang and et al., 2023] es una referencia explícita en esta línea, con *prompts* que inducen *chain-of-thought* adaptado a *text-to-SQL*. ODIS [Chang and Fosler-Lussier, 202 DAIL-SQL [D. Gao and et al., 2023], PET-SQL [Li and et al., 2024], SuperSQL [B. Li and et al., 2024], CoE-SQL [H. Zhang and et al., 2024] y CRED-SQL [Shaoming Duan, 2025] adoptan variantes del enfoque. La principal limitación es que la traza textual no es verificable estructuralmente; el modelo puede producir un razonamiento aparentemente válido y aun así generar una consulta errónea.

3.6.3. Razonamiento descompositivo

La pregunta se divide explícitamente en sub-preguntas, cada una de las cuales se resuelve por separado y se ensambla luego. DIN-SQL [Pourreza and Rafiei, 2023] es la referencia principal en esta línea. MAC-SQL [B. Wang and et al., 2023], CHESS [S. Talaei and et al., 2024], DEA-SQL [Xie and et al., 2024], RSL-SQL [et al., 2024], TA-SQL [Qu et al., 2024], SuperSQL [B. Li and et al., 2024] y SteinerSQL [Xutao Mao, 2025] la adoptan en variantes propias. La descomposición es coherente con la observación práctica de que las consultas complejas se resuelven mejor cuando se atacan por partes, aunque introduce el riesgo de descomposiciones incorrectas que arrastran errores difíciles de localizar.

3.6.4. Razonamiento de múltiples caminos

El sistema produce varias trayectorias o varios candidatos en paralelo y elige entre ellos. CHASE-SQL [M. Pourreza and et al., 2025], XiYan-SQL [Yingqi Gao, 2024], MetaSQL [Y. Fan and et al., 2024], PARSQL [Yaxun Dai, 2025], G^{3}R [**?** line:739] y N-best List Rerankers [L. Zeng and Hakkani-Tur, 2022] pertenecen a esta sub-categoría. La motivación está sustentada por la observación empírica de que el candidato más probable

según el modelo no es siempre el correcto; la diversidad del conjunto generado actúa como mecanismo de tolerancia a errores. La dificultad reside en construir un seleccionador o reordenador que efectivamente identifique el candidato correcto.

3.6.5. Razonamiento de retroalimentación iterativa

La consulta se refina en sucesivas iteraciones a partir de retroalimentación de ejecución, validación o un agente verificador. CHESS [S. Talaei and et al., 2024] con su Unit Tester, MAC-SQL [B. Wang and et al., 2023] con su agente de refinamiento, OpenSearch-SQL [X. Xie and Guo, 2025], DIN-SQL [Pourreza and Rafiei, 2023], RSL-SQL [et al., 2024], DEA-SQL [Xie and et al., 2024] y CHASE-SQL [M. Pourreza and et al., 2025] operan en ciclos iterativos. ErrorLLM [Hong et al., 2026] y SQLHD [Yang et al., 2025] proveen modelos de detección de errores que pueden alimentar tales ciclos. AmbiSQL [Ding et al., 2026] aborda iterativamente la resolución de ambigüedades en interacción con el usuario. La efectividad del paradigma depende de que la retroalimentación sea informativa y discriminante; en su ausencia, el ciclo puede oscilar sin converger.

3.6.6. Observaciones críticas

La descomposición y la retroalimentación iterativa han desplazado al razonamiento directo en los sistemas más recientes basados en LLMs. Sin embargo, la cadena de pensamiento y la descomposición, en sus implementaciones predominantes, operan sobre texto libre y son por lo tanto difíciles de auditar y de verificar formalmente. La investigación de esta tesis explora una estrategia complementaria: la descomposición y la retroalimentación operan sobre una RI estructurada y tipada, lo que convierte cada paso del razonamiento en un artefacto inspectable y verificable mediante reglas explícitas.

3.7. Propiedades Evaluativas Objetivo

Esta sección caracteriza la literatura por las propiedades del sistema que cada trabajo prioriza explícitamente en su evaluación. Se distinguen seis propiedades, no excluyentes, que guían el diseño y la experimentación.

3.7.1. Corrección

La corrección, medida típicamente como exactitud de ejecución (*execution accuracy*) o coincidencia exacta (*exact match*), es la propiedad históricamente dominante. Prácticamente todos los trabajos analizados la reportan: Spider [Lei et al., 2025], BIRD y otros benchmarks han consolidado su uso como métrica de referencia. La crítica recurrente, presente en EvaluatingRobustness [Fürst et al., 2024] y en NLinterfaces [Quamar et al., 2022], es que la exactitud sobre benchmarks controlados sobreestima el desempeño en entornos reales.

3.7.2. Eficiencia

Una fracción menor de trabajos prioriza explícitamente la eficiencia de la consulta generada, no solo la corrección de su resultado. CodeScore [Dong et al., 2024] y atscale2024sml [AtScale, Inc., 2024] contemplan métricas asociadas. SuperSQL [B. Li and et al., 2024], PET-SQL [Li and et al., 2024] y FinSQL [Zhang and et al., 2024] reportan parcialmente costos de ejecución. La VES (*Valid Efficiency Score*) introducida en BIRD ha promovido un mayor interés en la propiedad. Su relevancia es directa en entornos empresariales, donde dos consultas correctas pueden diferir órdenes de magnitud en costo de ejecución.

3.7.3. Robustez

Pertenecen a esta categoría los trabajos que evalúan el desempeño bajo perturbaciones del esquema, paráfrasis de la pregunta o cambios en el dominio. EvaluatingRobustness [Fürst et al., 2024], ROUTE [Y. Qin and et al., 2025], SQLHD [Yang et al., 2025], ErrorLLM [Hong et al., 2026] y AmbiSQL [Ding et al., 2026] adoptan la robustez como propiedad central. Spider 2.0 [Lei et al., 2025] desplaza la evaluación hacia escenarios empresariales donde la robustez se vuelve crítica. La consolidación de esta dimensión es todavía incipiente: no existe un protocolo único de evaluación de robustez aceptado por la comunidad.

3.7.4. Adaptabilidad

La adaptabilidad mide la capacidad del sistema para generalizar a nuevos dominios o esquemas no vistos durante el entrenamiento. ZeroNL2SQL [Z. Gu and et al., 2024], FinSQL [Zhang and et al., 2024], SchemaGraphSQL [Safdarian et al., 2025] y CRED-SQL [Shaoming Du priorizan explícitamente esta dimensión. La preocupación es relevante en entornos empresariales donde el esquema cambia con el tiempo y donde no es viable reentrenar el sistema con cada cambio.

3.7.5. Escalabilidad

La escalabilidad considera el comportamiento del sistema ante esquemas masivos, bases de datos con miles de tablas o cargas de trabajo sostenidas. CRUSH4SQL [et al., 2023], llmfk [Tang et al., 2026], CodeS [H. Li and et al., 2024] y CHASE-SQL [M. Pourreza and et al., 2025] la abordan parcialmente. La intersección con eficiencia es directa, pero la escalabilidad introduce además consideraciones sobre la ventana de contexto del modelo y sobre el costo amortizado del *schema linking*.

3.7.6. Interpretabilidad

La interpretabilidad se refiere a la posibilidad de inspeccionar y explicar el proceso que conduce a la consulta generada. Los trabajos que adoptan RI explícita —IRNet [Guo et al., 2019], NatSQL [Gan and et al., 2023]— y los que producen trazas estructuradas —DIN-SQL [Pourreza and Rafiei, 2023], CHESS [S. Talaei and et al., 2024], MAC-SQL [B. Wang and et al., 2023]— ofrecen mayor interpretabilidad por construcción. La propiedad cobra relevancia particular en aplicaciones empresariales sujetas a auditoría y gobernanza, contexto en el que la generación opaca de un sistema *end-to-end* resulta difícilmente admisible.

3.7.7. Observaciones críticas

La literatura presenta un sesgo claro hacia la corrección como propiedad casi excluyente, en detrimento de eficiencia, robustez, adaptabilidad y, sobre todo, interpretabilidad. Spider 2.0 y BIRD han contribuido a redirigir parte de la atención hacia robustez y eficiencia, pero la interpretabilidad sigue tratada de forma marginal. Para el contexto empresarial al que se orienta esta tesis, la interpretabilidad es una propiedad de primer orden: una consulta correcta pero no inspectable es difícil de operar de forma confiable. Esta consideración refuerza la decisión arquitectónica de adoptar una RI explícita y un bucle de verificación auditable.

3.8. Síntesis y Brecha de Investigación

En las secciones anteriores se ha analizado la literatura reciente bajo una taxonomía multidimensional que cubre el paradigma de aprendizaje, la arquitectura del sistema, las estrategias de representación, los mecanismos de enriquecimiento contextual, el paradigma de razonamiento y las propiedades evaluativas objetivo. La revisión permite extraer cinco observaciones de síntesis.

1. **Migración del aprendizaje al diseño arquitectónico.** Los trabajos más recientes han desplazado el énfasis del *fine-tuning* de extremo a extremo hacia configuraciones *in-context* e híbridas, complementadas con arquitecturas modulares y multiagente. La calidad final del sistema depende cada vez menos del ajuste del modelo base y cada vez más de la orquestación de componentes.

2. **Predominio de la decodificación libre sobre la representación intermedia.** La mayoría de los sistemas basados en LLMs prescinde de RI explícita y delega la corrección estructural al generador. Solo una minoría —IRNet [Guo et al., 2019], NatSQL [Gan and et al., 202 SmBoP [Rubin and Berant, 2021], PICARD [Scholak and et al., 2022], SQLFormer [A. Bazaga an SteinerSQL [Xutao Mao, 2025]— adopta variantes de RI o decodificación restringida. Esta minoritariedad contrasta con la creciente exigencia de verificabilidad en entornos empresariales.

3. **Auto-refinamiento sobre texto libre.** La mayoría de los bucles de refinamiento opera sobre la consulta como texto y sobre retroalimentación textual del error de ejecución. Esta granularidad limita la precisión de la corrección y favorece oscilaciones. La retroalimentación estructurada sobre una RI tipada permanece como una vía explorada solo parcialmente.

4. **Schema linking saturado, conocimiento externo incipiente.** El *schema linking* ha recibido atención sostenida y ha alcanzado un grado de madurez considerable. En contraste, la integración con conocimiento externo de dominio —glosarios, métricas, restricciones de negocio— sigue siendo una capacidad incipiente, sin formato estandarizado y sin integración nativa con la mayoría de las arquitecturas.

5. **Cobertura nula de consultas híbridas SQL/PGQ.** Pese a la formalización del estándar ISO/IEC 9075-16:2023 [ISO/IEC JTC 1/SC 32, 2023] y a la disponibilidad de motores que lo implementan [Wolde et al., 2023], ninguno de los sistemas analizados aborda la generación de consultas híbridas bajo este estándar. STRuCT-LLM [Stoisser et al., 2025] y text2gqlbench [Lyu et al., 2026] se aproximan al razonamiento sobre grafos, pero no contemplan la dualidad relacional/grafo propia de SQL/PGQ. Otros trabajos como SchemaGraphSQL [Safdarian et al., 2025] se sitúan en la frontera, pero sin abordar la generación bajo el estándar híbrido.

De la conjunción de estas cinco observaciones se desprende una brecha clara. La literatura ha desarrollado de forma extensiva técnicas parciales para mitigar alucinaciones en *text-to-SQL*: *schema linking*, decodificación restringida, descomposición, auto-refinamiento y verificación por ejecución. Sin embargo, no existe en la literatura analizada una arquitectura que integre simultáneamente: (i) una RI explícita y tipada que cubra tanto el componente relacional como el de patrón de grafo del estándar híbrido SQL/PGQ; (ii) una decodificación restringida o composicional sobre dicha RI; (iii) un bucle de verificación que opere a nivel de RI con retroalimentación estructurada en lugar de texto libre; y (iv) propiedades evaluativas que prioricen la interpretabilidad y la robustez junto con la corrección. El siguiente capítulo presenta la propuesta de la tesis, cuyo objetivo es articular estos cuatro componentes en una arquitectura coherente que aborde explícitamente la generación confiable de consultas SQL/PGQ a partir de lenguaje natural.

Capítulo 4

Propuesta: Arquitectura basada en Representación Intermedia para Generación Confiable de Consultas SQL/PGQ

Al concluir la revisión de la literatura actual, se evidencia una brecha de investigación claramente formulada: La literatura dispone de piezas maduras para abordar facetas individuales del problema *text-to-query*, entre ellas la descomposición del proceso de generación en etapas, la decodificación sujeta a restricciones, la verificación por ejecución y el enlazado robusto con el esquema.

No obstante, tales mecanismos han sido estudiados predominantemente de manera fragmentada, sin una articulación sistémica aplicada a consultas híbridas que combinen el paradigma relacional con patrones de grafo conforme al estándar SQL/PGQ [ISO/IEC JTC 1/SC 32, 202 A partir de esta constatación, el presente capítulo desarrolla la propuesta de investigación orientada a cerrar dicha brecha mediante una integración arquitectónica de estos aportes en el contexto de la generación de consultas empresariales sobre esquemas híbridos.

La propuesta se organiza en cinco bloques articulados entre sí. Primero, se establecen los requisitos y principios de diseño que debe satisfacer la arquitectura (Sección 4.1). Luego, se describe el flujo metodológico general del *pipeline*, así como la relación entre sus distintas etapas (Sección 4.2). A continuación, se detalla la arquitectura propuesta junto con sus componentes centrales, incluyendo la RI tipada, el anclaje al esquema, el compilador a SQL/PGQ y el bucle de verificación (Sección 4.3). Posteriormente, se presenta la estrategia de implementación sobre la base experimental seleccionada, distinguiendo los componentes reutilizados, los adaptados y los desarrollados específicamente para esta investigación (Sección 4.4). Por último, se fundamentan las decisiones de diseño adoptadas y se discuten las alternativas evaluadas (Sección 4.5).

4.1. Requisitos y principios de diseño

Los requisitos que se formulan en esta sección no constituyen una enumeración arbitraria, sino la traducción operacional de tres insumos previos del documento: (i) el planteamiento del problema desarrollado en el Capítulo 1, (ii) los fundamentos conceptuales revisados en el Capítulo 2, y (iii) las brechas identificadas en la síntesis del estado del arte. Cada requisito se identifica mediante un código (R_i) y se asocia con la dimensión del problema a la que responde.

R1. Anclaje estructural al esquema dual. La arquitectura debe producir consultas cuya estructura pueda verificarse contra un esquema dual compuesto por: (a) el esquema relacional, que incluye tablas, columnas, claves foráneas y tipos; y (b) el grafo de propiedad superpuesto, que comprende la definición de vértices, aristas, etiquetas y propiedades clave–valor conforme a la declaración `CREATE PROPERTY GRAPH` del estándar [ISO/IEC JTC 1/SC 32, 2023, Wolde et al., 2023]. Dicha verificabilidad debe ser posible antes de la ejecución de la consulta, y no solo como consecuencia de un error retornado por el motor.

R2. Representación intermedia tipada. La generación no debe producir SQL/PGQ de manera directa a partir del lenguaje natural. Debe mediar una RI explícita y tipada, cuya correctitud estructural sea decidible mediante reglas locales y cuya compilación a SQL/PGQ sea determinista. Este requisito traslada al contexto dual la lección central de IRNet [Guo et al., 2019] y RESDSQL [H. Li and Chen, 2023].

R3. Separación entre generación estadística y compilación determinista. La arquitectura debe preservar una separación estricta entre: (a) el componente estadístico encargado de producir hipótesis de interpretación a partir del lenguaje natural; y (b) el componente determinista responsable de compilar dichas hipótesis a SQL/PGQ. El propósito de esta separación es confinar las posibles alucinaciones del modelo al espacio de la RI, de modo que puedan detectarse y rechazarse antes de alcanzar el motor de ejecución.

R4. Verificación en dos regímenes. La propuesta debe incorporar un bucle de verificación que combine: (a) verificación estática sobre la RI y sobre la SQL/PGQ compilada, incluyendo referencias a elementos existentes, consistencia de tipos y coherencia entre los componentes relacionales y de grafo; y (b) verificación dinámica mediante ejecución controlada en un entorno de validación con DuckPGQ como motor de referencia [Wolde et al., 2023]. Este requisito retoma las propuestas de verificación revisadas en la Sección line:68 **??** y amplía su alcance al dominio híbrido.

R5. Modularidad y agnosticismo respecto del LLM. Los componentes de generación deben acoplarse al LLM mediante una interfaz abstracta que permita sustituir el modelo subyacente sin modificar la arquitectura general. Este requisito responde a la observación empírica de que los resultados reportados en el estado del arte dependen de manera crítica

del LLM utilizado, fenómeno documentado por SuperSQL [B. Li and et al., 2024] y por la proliferación de enfoques que combinan múltiples generadores [M. Pourreza and et al., 2025, Yingqi Gao, 2024].

R6. Trazabilidad de decisiones intermedias. Cada consulta generada debe poder rastrearse a lo largo de sus etapas intermedias: interpretación del lenguaje natural, construcción de la RI, compilación y verificación. La trazabilidad constituye un requisito previo para la interpretabilidad en entornos empresariales, una propiedad que el estado del arte actual aborda solo de manera marginal [Adedeji and Mabere, 2023, Quamar et al., 2022].

A partir de estos seis requisitos se derivan cinco *principios de diseño* que orientan las decisiones arquitectónicas concretas de la propuesta.

P1. Grounding antes que generación. Toda generación debe estar precedida por una fase explícita de anclaje al esquema dual. El modelo no genera SQL/PGQ sobre un esquema opaco, sino sobre una representación previamente proyectada y validada.

P2. Mediación obligatoria por representación intermedia. Ningún artefacto sintáctico de consulta alcanza el motor de ejecución sin pasar por la RI tipada. La RI constituye la frontera formal entre el componente estadístico y el componente determinista; la traducción desde la RI hacia SQL/PGQ es una función determinista, sin intervención adicional del modelo.

P3. Verificación en cascada. La validación de las consultas se organiza en niveles de costo creciente: la verificación estática precede a la dinámica, y solo las consultas que superan el nivel anterior son sometidas al siguiente. Esta progresión reduce el costo computacional y minimiza efectos colaterales sobre el RDBMS.

P4. Transparencia estructural. Cada componente debe exponer su entrada, su salida y su criterio de decisión en un formato auditable. De este modo, la opacidad propia del *prompting* monolítico se sustituye por una traza estructurada e inspeccionable.

P5. Desacoplamiento del modelo generador. El LLM se trata como una dependencia sustituible: los componentes de generación lo invocan a través de una interfaz abstracta, y ninguna decisión arquitectónica queda atada a un modelo específico.

Estos principios se concretan en el *pipeline* descrito en la siguiente sección.

4.2. Flujo metodológico general

El *pipeline* propuesto se articula en cinco etapas secuenciales, junto con un bucle de refinamiento asimétrico entre la construcción de la RI y la fase de verificación. La Figura 4.1 resume el flujo metodológico general de la propuesta.

1. **Pre-procesamiento y anclaje al esquema dual.** Esta etapa recibe como entrada la pregunta en lenguaje natural, el esquema relacional y la definición del grafo de

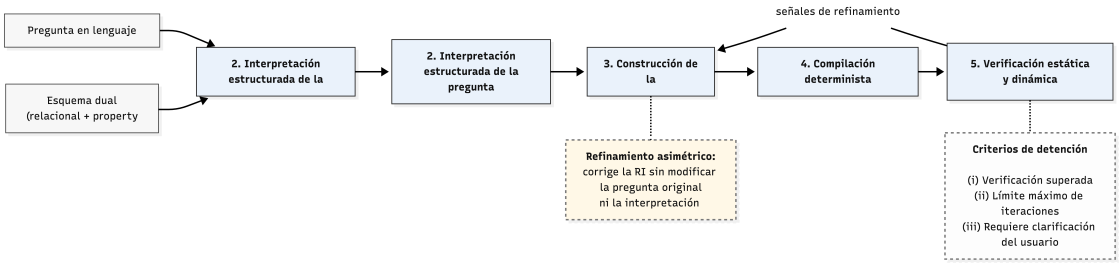

Figura 4.1: Flujo metodológico general del *pipeline* propuesto. Las cinco etapas secuenciales procesan la pregunta en lenguaje natural y el esquema dual (relacional y grafo de propiedad) hasta producir una consulta SQL/PGQ verificada.

propiedad. Su salida es un esquema proyectado, es decir, un subconjunto del esquema dual enriquecido con señales de relevancia, tales como tablas y columnas candidatas, vértices y aristas potencialmente implicados, así como *cell-values* relevantes. Esta fase materializa el principio P1 y se apoya en ideas de anclaje al esquema y reducción del espacio de búsqueda reportadas en [et al., 2024, S. Talaei and et al., 2024].

2. **Interpretación estructurada de la pregunta.** A partir de la pregunta y del esquema proyectado, esta etapa produce una descomposición explícita de la intención del usuario en componentes interpretables: entidades mencionadas, restricciones, proyecciones esperadas y tipo de razonamiento requerido. En particular, identifica si la consulta demanda razonamiento relacional, razonamiento sobre grafos o una combinación de ambos. Esta etapa se inspira en el paradigma de descomposición presentado en [Pourreza and Rafiei, 2023, B. Wang and et al., 2023], extendiéndolo al contexto híbrido SQL/PGQ.

3. **Construcción de la representación intermedia tipada.** Con base en la interpretación estructurada, se construye una instancia de la RI propuesta. Esta etapa constituye el núcleo conceptual de la arquitectura, pues traduce la intención del usuario a una forma estructurada, tipada y validable antes de cualquier compilación al lenguaje de consulta final.

4. **Compilación determinista a SQL/PGQ.** Una vez que la RI satisface sus restricciones estructurales, se compila a una consulta concreta mediante reglas deterministas, sin intervención adicional del LLM. De este modo, la generación estadística queda separada de la síntesis sintáctica final, y la consulta producida se ajusta por construcción a la gramática del estándar objetivo.

5. **Verificación estática y dinámica.** La consulta compilada se somete, en primer lugar, a una verificación estática orientada a comprobar su consistencia estructural respecto del esquema dual. Si supera esta fase, se procede a una verificación dinámica mediante ejecución controlada sobre DuckPGQ [Wolde et al., 2023]. Los errores detectados en cualquiera de estos dos niveles se transforman en señales de refinamiento que retroalimentan la etapa de construcción de la RI.

El bucle de refinamiento es asimétrico en el sentido que la retroalimentación no

modifica la pregunta original ni reabre la interpretación de alto nivel de la etapa dos, sino que actúa exclusivamente sobre la construcción de la RI. Esta decisión responde al principio P2, ya que concentra la corrección en el espacio donde la estructura puede manipularse de forma explícita y verificable mediante reglas. El refinamiento concluye cuando se satisface alguno de los siguientes criterios: (i) la verificación es superada, (ii) se alcanza un número máximo de iteraciones, o (iii) el sistema determina que la pregunta presenta una ambigüedad intrínseca que requiere clarificación del usuario, en consonancia con lo planteado por [Ding et al., 2026].

4.3. Arquitectura y componentes

4.4. Implementación sobre la base experimental

4.5. Justificación de decisiones de diseño

4.6. Síntesis y transición

Capítulo 5

Experimentos

Capítulo 6

Resultados

Capítulo 7

Conclusiones y Trabajos Futuros

Las conclusiones de la tesis son una parte muy importante y tiene las siguientes partes.

Escribir las conclusiones de su trabajo por cada uno de los objetivos específicos definidos en el Capítulo 1 y luego escribir la conclusión general de tu trabajo.

7.1. Problemas encontrados

La segunda parte de este capítulo corresponde a los problemas encontrados, esta sección es muy importante para que los siguientes estudiantes o investigadores que hagan algo en esta línea no cometan los mismos errores y tu tesis sea un buen peldaño para avanzar más rápido.

7.2. Recomendaciones

En esta sección el tesista debe reflejar que la tesis ha permitido adquirir nuevos conocimientos que podrían servir para guiar otros trabajos en el futuro.

7.3. Trabajos futuros

En base a los puntos anteriores es recomendable que tu tesis también sugiera trabajos futuros. Esta sección es esecialmente útil para otras ideas de tesis.

Todo este capítulo no debe ser más de unas 4 páginas.

Bibliografía

[A. Bazaga and et al., 2023] A. Bazaga, P. L. and et al. (2023). Sqlformer: Deep autoregressive query graph generation for text-to-sql translation.

[Adedeji and Mabere, 2023] Adedeji and Mabere, M. (2023). From data to decisions: Llms redefining business intelligence author. *Journal of Artificial Intelligence*, 6:14.

[Aho et al., 2006] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition.

[Angles et al., 2017] Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., and Vrgoč, D. (2017). Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5).

[AtScale, Inc., 2024] AtScale, Inc. (2024). *Semantic Modeling Language (SML) Specification v0.1*. AtScale. Technical Specification for Open Source Semantic Modeling.

[B. Li and et al., 2024] B. Li, Y. Luo, C. C. and et al. (2024). The dawn of natural language to sql: Are we fully ready?

[B. Li and Luo, 2025] B. Li, J. Zhang, J. F. Y. X. C. C. N. T. and Luo, Y. (2025). Alpha-sql: Zero-shot text-to-sql using monte carlo tree search. *ICML 2025*.

[B. Wang and et al., 2023] B. Wang, C. Ren, J. Y. and et al. (2023). Mac-sql: A multi-agent collaborative framework for text-to-sql. *COLING 2025*.

[B. Wang and et al., 2021] B. Wang, R. Shin, X. L. and et al. (2021). Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. *ACL 2020*.

[Brown et al., 2020] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[C. Gao and et al., 2022] C. Gao, B. Li, W. Z. and et al. (2022). Towards generalizable and robust text-to-sql parsing. *EMNLP 2022 Findings*.

[Chang and Fosler-Lussier, 2023] Chang, S. and Fosler-Lussier, E. (2023). Selective demonstrations for cross-domain text-to-sql. *Findings of EMNLP 2023*.

[Chen et al., 1998] Chen, T. Y., Cheung, S. C., and Yiu, S. M. (1998). Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology.

[Chi, 2025] Chi, H. (2025). Reducing hallucinations in text-to-sql building trust and accuracy in data access.

[Chomsky, 1956] Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3):113–124.

[Codd, 1970] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

[D. Gao and et al., 2023] D. Gao, H. Wang, Y. L. and et al. (2023). Text-to-sql empowered by large language models: A benchmark evaluation. *PVLDB 2024*.

[Ding et al., 2026] Ding, Z., Lin, Y., Zeng, T., Zhu, R., Ding, B., and Zhou, J. (2026). Ambisql: Interactive ambiguity detection and resolution for text-to-sql.

[Dong et al., 2024] Dong, Y., Ding, J., Jiang, X., Li, G., Li, Z., and Jin, Z. (2024). Codescore: Evaluating code generation by learning code execution.

[et al., 2023] et al., M. K. (2023). Crush4sql: Collective retrieval using schema hallucination for text2sql. *EMNLP 2023*.

[et al., 2025] et al., S. G. (2025). Blendsql: A scalable dialect for unifying hybrid question answering in relational algebra. *ACL 2024 Findings*.

[et al., 2024] et al., Z. C. (2024). Rsl-sql: Robust schema linking in text-to-sql generation.

[Fürst et al., 2024] Fürst, J., Kosten, C., Nooralahzadeh, F., Zhang, Y., and Stockinger, K. (2024). Evaluating the data model robustness of text-to-sql systems based on real user queries.

[Gan and et al., 2023] Gan, Y. and et al. (2023). Improving generalization in language model-based text-to-sql semantic parsing: Two simple semantic boundary-based techniques. *ACL 2023*.

[Guo et al., 2019] Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., and Zhang, D. (2019). Towards complex text-to-sql in cross-domain database with intermediate representation.

[H. Fu and et al., 2023] H. Fu, C. Liu, B. W. and et al. (2023). Catsql : Towards real world natural language to sql applications. *PVLDB 2023*.

[H. Li and Chen, 2023] H. Li, J. Zhang, C. L. and Chen, H. (2023). Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. *AAAI 2023*.

[H. Li and et al., 2024] H. Li, J. Zhang, H. L. and et al. (2024). Codes: Towards building open-source language models for text-to-sql. *SIGMOD 2024*.

[H. Zhang and et al., 2024] H. Zhang, R. Cao, H. X. and et al. (2024). Coe-sql: In-context learning for multi-turn text-to-sql with chain-of-editions. *NAACL 2024*.

[Hong et al., 2026] Hong, Z., Chen, H., Yuan, Z., Zhang, Q., Zhuang, L., Liao, Q., Huang, F., Song, Y., and Huang, X. (2026). Errorllm: Modeling sql errors for text-to-sql refinement.

[Hui and et al., 2022] Hui, B. and et al. (2022). S2sql: Injecting syntax to question-schema interaction graph encoder for text-to-sql parsers. *ACL 2022 Findings*.

[ISO/IEC JTC 1/SC 32, 2023] ISO/IEC JTC 1/SC 32 (2023). Information technology – database languages – sql – part 16: Property graph queries (sql/pgq). Standard, International Organization for Standardization, Geneva, CH.

[J. Huang and et al., 2021] J. Huang, Y. Wang, Y. W. and et al. (2021). Relation aware semi-autoregressive semantic parsing for nl2sql.

[J. Qi and et al., 2022] J. Qi, J. Tang, Z. H. and et al. (2022). Rasat: Integrating relational structures into pretrained seq2seq model for text-to-sql. *EMNLP 2022*.

[J. Shi and et al., 2025] J. Shi, Bo Xu, J. L. and et al. (2025). Gen-sql: Efficient text-to-sql by bridging natural language question and database schema with pseudo-schema. *COLING 2025*.

[Ji et al., 2023] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12).

[L. Dou and et al., 2022] L. Dou, Y. Gao, X. L. and et al. (2022). Importance of synthesizing high- quality data for text-to-sql parsing.

[L. Zeng and Hakkani-Tur, 2022] L. Zeng, S. H. K. P. and Hakkani-Tur, D. (2022). N-best hypothe- ses reranking for text-to-sql systems.

[Lei et al., 2025] Lei, F., Su, J., Chen, Y., Li, Q., Zhang, Z.-X., et al. (2025). Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. Oral Presentation.

[Li and et al., 2022] Li, J. and et al. (2022). Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing.

[Li et al., 2023] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C.-C., Huang, F., Cheng, R., and Li, Y. (2023). Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[Li and et al., 2024] Li, Z. and et al. (2024). Pet-sql: A prompt-enhanced two-round refinement of text-to-sql with cross-consistency.

[Lin and et al., 2020] Lin, X. V. and et al. (2020). Bridging textual and tabular data for cross-domain text-to-sql semantic parsing.

[Liu et al., 2025] Liu, R., Chen, X., Zhang, J., Zhang, Q., Zhang, Y., and Yang, B. (2025). Safenlidb: A privacy-preserving safety alignment framework for llm-based natural language database interfaces.

[Lyu et al., 2026] Lyu, S., Ban, L., Wu, Z., Luo, T., Liu, J., Ma, C., Luo, Y., Tang, N., Qi, S., Lin, H., Liu, Y., and Hong, C. (2026). Text2gql-bench: A text to graph query language benchmark [experiment, analysis & benchmark].

[M. Pourreza and et al., 2025] M. Pourreza, H. Li, R. S. and et al. (2025). Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *ICLR 2025*.

[Pérez et al., 2025] Pérez, A. S., Boukhary, A., Papotti, P., Lozano, L. C., and Elwood, A. (2025). An LLM-based approach for insight generation in data analysis. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 562–582, Albuquerque, New Mexico. Association for Computational Linguistics.

[Pierce, 2002] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.

[Pourreza and Rafiei, 2023] Pourreza, M. and Rafiei, D. (2023). Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *NeurIPS 2023*.

[Pourreza and Rafiei, 2024] Pourreza, M. and Rafiei, D. (2024). Dts-sql: Decomposed text-to-sql with small large language models. *EMNLP 2024 Findings*.

[Qinggang Zhang, 2025] Qinggang Zhang, Hao Chen, J. D. S. C. F. H. X. H. (2025). Structure guided large language model for sql generation. *ICML 2024*.

[Qu et al., 2024] Qu, G., Li, J., Li, B., et al. (2024). Before generation, align it! a novel and effective strategy for mitigating hallucinations in text-to-sql generation. *ACL 2024 Findings*.

[Quamar et al., 2022] Quamar, A., Efthymiou, V., Lei, C., and Lei, C. (2022). Natural language interfaces to data. *Foundations and Trends in Databases*, 11(4):319–414.

[Rubin and Berant, 2021] Rubin, O. and Berant, J. (2021). Smbop: Semi-autoregressive bottom-up semantic parsing. *NAACL 2021*.

[S. Talaei and et al., 2024] S. Talaei, M. Pourreza, Y.-C. C. and et al. (2024). Chess: Contextual harnessing for efficient sql synthesis. *ICML*.

[Safdarian et al., 2025] Safdarian, A., Mohammadi, M., Jahanbakhsh, E., Naderi, M. S., and Faili, H. (2025). Schemagraphsql: Efficient schema linking with pathfinding graph algorithms for text-to-sql on large-scale databases.

[Scholak and et al., 2022] Scholak, T. and et al. (2022). Picard: Parsing incrementally for constrained auto-regressive decoding from language models. *EMNLP 2021*.

[Shaoming Duan, 2025] Shaoming Duan, Zirui Wang, C. L. Z. Z. Y. Z. P. H. L. Y. Z. P. (2025). Cred-sql: Enhancing large scale database text-to-sql.

[Stoisser et al., 2025] Stoisser, J. L., Martell, M. B., Phillips, L., Hansen, C., and Fauqueur, J. (2025). Struct-llm: Unifying tabular and graph reasoning with reinforcement learning for semantic parsing. *ICLR 2025*.

[T. Ren and et al., 2024] T. Ren, Y. Fan, Z. H. and et al. (2024). Purple: Making a large language model a better sql writer. *ICDE 2024*.

[Tang et al., 2026] Tang, Z., Zhang, Y., Cai, S., and Wang, R. (2026). Llm-fk: Multi-agent llm reasoning for foreign key detection in large-scale complex databases.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[Victor Zhong, 2017] Victor Zhong, Caiming Xiong, R. S. (2017). Seq2sql: Generating structured queries from natural language using reinforcement learning.

[Wenbo Xu, 2025] Wenbo Xu, Haifeng Zhu, L. Y. C. L. P. H. S. D. J. Z. P. (2025). Ts-sql: Test-driven self-refinement for text-to-sql. *EMNLP 2025 Findings*.

[Wolde et al., 2023] Wolde, D. t., Szárnyas, G., and Boncz, P. (2023). Duckpgq: Bringing sql/pgq to duckdb. *Proc. VLDB Endow.*, 16(12):4034–4037.

[X. Dong and et al., 2023] X. Dong, C. Zhang, Y. G. and et al. (2023). C3: Zero-shot text-to-sql with chatgpt.

[X. Xie and Guo, 2025] X. Xie, G. Xu, L. Z. and Guo, R. (2025). Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment. *ACM on Management of Data*.

[Xie and et al., 2024] Xie, Y. and et al. (2024). Decomposition for enhancing attention: Improving llm-based text-to-sql through workflow paradigm. *ACL 2024 Findings*.

[Xutao Mao, 2025] Xutao Mao, Tao Liu, H. Z. (2025). Steinersql: Graph-guided mathematical reasoning for text-to-sql generation.

[Y. Fan and et al., 2024] Y. Fan, Z. He, T. R. and et al. (2024). Metasql: A generate-then-rank framework for natural language to sql translation.

[Y. Qin and et al., 2025] Y. Qin, C. Chen, Z. F. and et al. (2025). Route: Robust multi-task tuning and collaboration for text-to-sql. *ICLR 2025*.

[Yang et al., 2025] Yang, B., Xia, Y., Sun, W., and Liu, Y. (2025). Hallucination detection for llm-based text-to-sql generation via two-stage metamorphic testing.

[Yaxun Dai, 2025] Yaxun Dai, Haiqin Yang, M. H. P. C. (2025). Parsql: Enhancing text-to-sql through parsing and reasoning. *ACL 2025 Findings*.

[Yingqi Gao, 2024] Yingqi Gao, Yifu Liu, X. L. X. S. Y. Z. Y. W. S. L. W. L. Y. H. Z. L. J. G. L. M. Y. L. (2024). Xiyan-sql: A multi-generator ensemble framework for text-to-sql.

[Z. Gu and et al., 2024] Z. Gu, J. F. and et al. (2024). Interleaving pre-trained language models and large language models for zero-shot nl2sql generation.

[Z. Gu and et al., 2023] Z. Gu, J. Fan, N. T. and et al. (2023). Select-sql: Self-correcting ensemble chain-of-thought for text-to-sql.

[Zelle and Mooney, 1996] Zelle, J. M. and Mooney, R. J. (1996). Learning to parse database queries using inductive logic programming. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI)*, pages 1050–1055.

[Zhang and et al., 2024] Zhang, C. and et al. (2024). Finsql: Model-agnostic llms-based text-to-sql framework for financial analysis.

[Zhang and et al., 2023] Zhang, H. and et al. (2023). Act-sql: In-context learning for text-to-sql with automatically-generated chain-of-thought. *Findings of EMNLP 2023*.